Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster en Computación Cuántica

Ventaja cuántica en ciencia de datos: clasificación, optimización y representación en régimen NISQ

Trabajo Fin de Máster

presentado por: Juan David Prada Malagón

Dirigido por: Ricardo Rafael Domínguez Reyes

Ciudad: Barcelona, España

Fecha: November 2, 2026

Índice general

Índice de figuras **iv**

Índice de cuadros **viii**

Resumen **ix**

Abstract **x**

1. Introducción **1**

1.1. Motivación y justificación del tema 1

1.2. Planteamiento del problema 1

1.3. Estructura del trabajo 2

2. Contexto y estado de la técnica **4**

2.1. Antecedentes históricos 4

2.2. Computación cuántica en la era NISQ 5

2.3. Aprendizaje automático cuántico 6

2.4. Optimización cuántica aplicada 7

2.5. Análisis crítico de los trabajos previos 8

3. Objetivos **9**

3.1. Objetivo general 9

3.2. Objetivos específicos 9

4. Metodología y desarrollo experimental **10**

4.1. Entorno tecnológico 10

4.2. Diseño experimental 10

4.3. Clasificación 11

4.3.1. Datasets 12

4.3.2. Preparación compartida 14

4.3.3. Representación en el espacio de Hilbert 15

4.3.4. Modelos 16

4.3.5. Métricas de clasificación y latencia 17

4.4. Optimización 17
4.4.1. Problemas e instancias 17
4.4.2. Preparación: de problema a QUBO/Ising 19
4.4.3. Solvers 20
4.4.4. Métricas de optimización y latencia 20
4.5. Comparación clásico vs cuántico 20
4.6. Reproducibilidad 22
4.7. Limitaciones del diseño 22

5. Resultados y análisis **23**
5.1. Clasificación 24
5.1.1. Iris 24
5.1.2. Bars and Stripes 31
5.1.3. Ising 35
5.1.4. Lectura de la familia 39
5.2. Optimización 39
5.2.1. TSP 39
5.2.2. Minimum dominating set 45
5.2.3. MaxCut 50
5.2.4. Lectura de la familia 56
5.3. Discusión: la representación como eje 56

6. Conclusiones **58**
6.1. Conclusiones 58
6.2. Cumplimiento de objetivos 59
6.3. Cierre 59

7. Líneas de trabajo futuro **60**

Bibliografía **61**

A. Apéndices **69**
A.1. Repositorio y catálogo de notebooks 69
A.2. Entorno de software 69
A.3. Datasets e instancias 70

A.4. Figuras complementarias de clasificación 71

Índice de figuras

1.1. Estructura del TFM. 3

2.1. Evolución del hardware cuántico en la era NISQ (2016–2024) 5

4.1. Pipeline experimental común: dato → preparación → modelo/solver → métricas agregadas. 11

4.2. Iris antes de los clasificadores. Proyección visual de las cuatro variables originales; el modelo no usa esta PCA. Círculos huecos: test del protocolo. . 12

4.3. Bars and Stripes antes de aplanar y antes de cualquier clasificador. Cada panel es un patrón 4×4; $y \in \{+1, -1\}$ es la clase del catálogo. 13

4.4. Bars and Stripes tras la reducción a componentes y antes de cualquier clasificador. Las dos clases comparten el mismo plano. Círculos huecos: test. . 13

4.5. Sistema de Ising: cadena 1D de cuatro espines con acoplamiento J entre vecinos y campo transversal h, en layout abierto o cerrado. Las features son las amplitudes del estado fundamental de este Hamiltoniano para cada h. . 14

4.6. Preparación compartida y bifurcación C/Q. Tras split, PCA (si $d > 4$) y MinMax$[0, \pi]$, el mismo X alimenta baselines clásicos; en la rama cuántica pasa antes por el embedding al espacio de Hilbert. Ambas ramas se evalúan con las mismas métricas de calidad y latencia. 15

4.7. Instancia de TSP antes de cualquier solver. Cinco nodos y las distancias de las aristas. 18

4.8. Instancia de MDS antes de cualquier solver. Seis vértices y seis aristas. . . . 18

4.9. Instancia de MaxCut antes de cualquier solver. Seis vértices y doce aristas; el corte óptimo es 8. 19

4.10. Optimización: la representación QUBO/Ising es compartida por SA, QAOA y VQE; BF/greedy/ILP operan sobre la formulación nativa o equivalente. . 20

5.1. Media de la familia clásica y de la cuántica sobre Iris, Bars and Stripes e Ising. El AUC cuántico no incluye el VQC, que no produce scores. 24

5.2. F1 medio de test frente al tiempo de entrenamiento (escala logarítmica). El SVM ocupa la esquina de alta calidad y bajo coste; QNN y VQC pagan el bucle variacional. 25

5.3. Ranking de F1 en Iris. El SVM encabeza la lista; el QSVM es el único que cae de forma clara. 26
5.4. Frontera del SVM en las dos primeras componentes de Iris. Los círculos marcan errores de test, concentrados en el solape de las dos clases no separables. 26
5.5. Frontera del Random Forest en Iris. La partición es local; el gap de 0,050 aparece donde las dos clases solapadas cambian de hoja. 27
5.6. Frontera de XGBoost en Iris. La calidad empata con el bosque; el tiempo de ajuste es el mayor de la rama clásica. 28
5.7. Frontera del QSVM (mapa ZZ) en Iris. El kernel fragmenta el plano; los errores no se concentran solo en el solape. 29
5.8. Frontera de la QNN en Iris. La cabeza lineal sobre $\langle Z_i \rangle$ produce regiones más compactas que el kernel ZZ. 30
5.9. Frontera del VQC en Iris. La calidad empata con los árboles; el tiempo de entrenamiento es dos órdenes por encima del SVM. 30
5.10. Ranking de F1 en Bars and Stripes. El SVM lidera; el VQC cierra la lista. . 31
5.11. Gap de generalización en Bars and Stripes. El Random Forest es el que más se separa entre train y test. 32
5.12. Frontera del SVM en Bars and Stripes. El plano mezcla las clases; el margen RBF es el que menos errores acumula. 32
5.13. Frontera del Random Forest en Bars and Stripes. La partición ajusta el train; el gap de 0,146 mide lo que no se sostiene en test. 33
5.14. Frontera de XGBoost en Bars and Stripes. El F1 queda justo bajo el SVM; el AUC es el más alto de la tabla. 34
5.15. Frontera del QSVM en Bars and Stripes. El kernel de fidelidad no ordena la nube mejor que el RBF y multiplica el tiempo de inferencia. 35
5.16. Frontera de la QNN en Bars and Stripes. El AUC empata con el SVM; la etiqueta dura no. 36
5.17. Frontera del VQC en Bars and Stripes. El corte deja la región de solape llena de errores de test. 36
5.18. Ranking de F1 en Ising. SVM y QSVM comparten el primer puesto. 37
5.19. Success rate medio frente al tiempo medio. La fuerza bruta es exacta y barata en estas instancias; QAOA y VQE se sitúan en segundos de simulación. 40

5.20. TSP: success rate frente al tiempo. El SA es el único que no satura el éxito; ILP, QAOA y VQE comparten la escala del segundo. 41
5.21. Ruta óptima del TSP por enumeración. Coste 16. 41
5.22. Ruta del greedy en el TSP. El coste coincide con el óptimo. 42
5.23. Ruta del ILP en el TSP. Óptimo de coste 16, con el mayor tiempo clásico de la tabla. 43
5.24. Una ruta óptima del simulated annealing, $[0, 4, 2, 3, 1, 0]$. El success rate sobre semillas es 0,90. 43
5.25. Ruta devuelta por QAOA en el TSP. Coste óptimo, en torno a un segundo de simulación. 44
5.26. Ruta devuelta por VQE en el TSP. Mismo coste óptimo, más evaluaciones de energía que QAOA. 45
5.27. MDS: success rate frente al tiempo. Greedy no acierta; QAOA acierta en una fracción de las semillas y es el más lento. 46
5.28. Conjunto dominante mínimo por enumeración. Tamaño 2. 46
5.29. Conjunto del greedy: Mario, Sarah y Ana. Tamaño 3, uno por encima del óptimo. 47
5.30. Conjunto dominante del ILP. Tamaño óptimo 2. 48
5.31. Conjunto devuelto por simulated annealing. Tamaño 2 en las semillas reportadas. 49
5.32. Una ejecución óptima de QAOA: Ana y Enrique. El success rate sobre semillas es 0,875. 49
5.33. Conjunto devuelto por VQE. Tamaño óptimo, por debajo del tiempo de QAOA y por encima del SA. 50
5.34. MaxCut: éxito 1 para todos los solvers. El eje de tiempo separa enumeración, greedy y SA del ILP y, más allá, de QAOA y VQE. 51
5.35. Corte óptimo de MaxCut por enumeración. Valor 8. 52
5.36. Corte del greedy en MaxCut. Valor óptimo 8. 53
5.37. Corte del ILP en MaxCut. Valor 8. 53
5.38. Corte del simulated annealing en MaxCut. Valor óptimo en las semillas reportadas. 54
5.39. Corte devuelto por QAOA. Valor óptimo 8, en torno a 1,2 s de simulación. . 55

5.40. Corte devuelto por VQE. Mismo óptimo que QAOA, con más evaluaciones de energía. 55

A.1. Accuracy en test por modelo y dataset, con la media de los tres. QSVM y VQC son los modelos con mayor variación entre datasets. 71
A.2. Métricas de test medias por modelo. El VQC no exporta probabilidades, por lo que no tiene AUC. 71
A.3. Tiempos medios de entrenamiento y predicción por modelo (escala logarítmica). 72
A.4. Brecha de generalización media (train − test) en accuracy y F1. 72

Índice de cuadros

4.1. Entorno software del estudio. Las versiones corresponden a las dependencias fijadas del repositorio. 10
4.2. Ejes del estudio. La representación no es una tercera familia de datos: es la transformación que comparten clasificación y optimización. 11
4.3. Métricas de calidad y latencia en clasificación. 17
4.4. Métricas de calidad y latencia en optimización. C es el coste o el corte obtenido; $C^\star$ es el óptimo de la instancia. 21
4.5. Ciclo de modelado comparado. La columna cualitativa delimita qué es común y qué es propio de cada familia; la cuantitativa aplica las métricas de calidad y latencia. 21

5.1. Iris, conjunto de test. Gap $= \mathrm{Acc}_{\mathrm{train}} - \mathrm{Acc}_{\mathrm{test}}$. 25
5.2. Bars and Stripes, conjunto de test. 31
5.3. Ising, conjunto de test. Recall macro del SVM y del QSVM: 0,979. 37
5.4. TSP. $C^\star = 16$. El SA promedia semillas; el resto alcanza el óptimo en las ejecuciones reportadas. 40
5.5. MDS. $C^\star = 2$. QAOA promedia semillas; el mejor coste de esas semillas sigue siendo 2. 45
5.6. MaxCut HamLib. $C^\star = 8$. Todos los solvers reportados alcanzan el corte óptimo. 51

6.1. Objetivos específicos, evidencia y conclusiones asociadas. Fuente: elaboración propia. 59

7.1. Líneas de trabajo futuro. Fuente: elaboración propia. 60

A.1. Organización del corpus experimental. Fuente: elaboración propia. 69
A.2. Versiones de las dependencias principales. Fuente: elaboración propia. . . . 70
A.3. Datasets de clasificación (train/test entre paréntesis) e instancias de optimización. Fuente: elaboración propia. 70

Resumen

Este Trabajo Fin de Máster (TFM) compara algoritmos de computación cuántica, algoritmos cuántico-inspirados y métodos clásicos en dos áreas de la ciencia de datos: clasificación y optimización combinatoria. Esas dos familias sirven aquí de punto de partida: a partir de ellas se pueden plantear, con el mismo criterio de comparación, otras aplicaciones como regresión lineal, clustering, series temporales, detección de anomalías, reinforcement learning, generativa o simulación de sistemas.

En clasificación se usan tres conjuntos de datos: Iris, Bars and Stripes e Ising unidimensional (clasificación de fases). Se comparan QSVM, QNN y VQC frente a SVM, Random Forest y XGBoost, con el mismo protocolo de partición, escalado y, cuando hace falta, PCA.

En optimización se resuelven tres problemas: el Problema del Viajante (TSP), el Minimum Dominating Set (MDS) y Maximum Cut sobre una instancia de HamLib. Se evalúan QAOA y VQE frente a fuerza bruta, greedy, ILP y Simulated Annealing sobre QUBO.

En las instancias y el régimen NISQ simulados aquí, los métodos clásicos igualan o superan a los cuánticos en precisión y en tiempo de cómputo. El aporte del trabajo es el protocolo de comparación y una lectura clara de hasta dónde llega (y no llega) la ventaja cuántica práctica en estos casos.

Palabras clave: computación cuántica, quantum-inspired, ciencia de datos, clasificación, optimización combinatoria, Iris, Bars and Stripes, Ising, TSP, MDS, MaxCut, QSVM, VQC, QAOA, VQE, NISQ.

Abstract

This Master's Thesis compares quantum computing algorithms, quantum-inspired methods and classical baselines on two areas of data science: classification and combinatorial optimisation. Those two families are the starting point: the same comparison criteria can later be taken to other applications such as linear regression, clustering, time series, anomaly detection, reinforcement learning, generative models or system simulation.

Classification uses three datasets: Iris, Bars and Stripes and one-dimensional Ising (phase classification). QSVM, QNN and VQC are compared with SVM, Random Forest and XGBoost under the same split, scaling and, when needed, PCA protocol.

Optimisation covers three problems: the Travelling Salesperson Problem (TSP), the Minimum Dominating Set (MDS) and Maximum Cut on a HamLib instance. QAOA and VQE are evaluated against brute force, greedy, ILP and Simulated Annealing on QUBO.

On the instances and simulated NISQ regime studied here, classical methods match or beat the quantum ones in accuracy and runtime. The contribution is the comparison protocol and a clear account of where practical quantum advantage does and does not appear in these cases.

Keywords: quantum computing, quantum-inspired, data science, classification, combinatorial optimisation, Iris, Bars and Stripes, Ising, TSP, MDS, MaxCut, QSVM, VQC, QAOA, VQE, NISQ.

1. Introducción

La computación cuántica y la ciencia de datos convergen en aprendizaje automático cuántico (QML) y optimización cuántico-inspirada. En la era NISQ, la ventaja práctica frente a métodos clásicos sigue sin estar demostrada de forma sistemática (Preskill, 2018; Biamonte et al., 2017). Los algoritmos variacionales (QAOA, VQE, VQC) y los kernels cuánticos son el enfoque habitual en hardware intermedio (Farhi et al., 2014; Cerezo et al., 2021; Havlíček et al., 2019); en demos pequeñas y con protocolo justo, los baselines clásicos suelen igualar o ganar (Bowles et al., 2024; Yu et al., 2026; Z-537-437, n.d.). El interés estratégico por la tecnología, visible en su consideración como asunto de seguridad nacional (Quantum Zeitgeist, n.d.c), hace más necesario separar lo medido de lo prometido.

Este TFM hace una comparación reproducible en **simulación clásica**: tres tareas de clasificación y tres de optimización combinatoria, con el mismo preproceso y con métricas de calidad y de tiempo. El marco usa tres ejes: *qué* (clasificar), *cómo* (optimizar) y *representación* (feature map / QUBO–Ising) (Lloyd et al., 2020; PennyLane, n.d.c; Parzygnat et al., 2025).

1.1 Motivación y justificación del tema

Clasificación y optimización son las dos familias con las que se construye el corpus. Con el mismo criterio de comparación se puede extender después a otras aplicaciones (regresión, clustering, series temporales, anomalías, reinforcement learning, generativa, simulación), pero esas no forman parte de los experimentos de este trabajo.

La teoría apunta aceleraciones en casos concretos (Grover, 1996; Shor, 1994) y, en QML, a kernels en espacios de características grandes (Havlíček et al., 2019; Schuld & Killoran, 2019). En la práctica la ventaja depende de la alineación dato–mapa (Huang et al., 2021) y queda limitada por pocos qubits, ruido y concentración del kernel (Preskill, 2018; Thanasilp et al., 2024; McClean et al., 2018). Hace falta medir precisión y wall-clock con el mismo protocolo, no demos asimétricas (Bowles et al., 2024).

1.2 Planteamiento del problema

Pregunta: *¿En las instancias y el régimen NISQ simulados aquí, los enfoques cuánticos o cuántico-inspirados mejoran la calidad o el esfuerzo frente a baselines clásicos?*

1. **Clasificación.** Datasets: Iris (scikit-learn developers, n.d.), Bars and Stripes (PennyLane, n.d.a,n) e Ising 1D (fases) (PennyLane, n.d.h,n). Modelos: SVM, Random Forest, XGBoost frente a QSVM, QNN, VQC (PennyLane, n.d.m). Mismo split, escalado MinMax$[0, \pi]$ y PCA a 4 cuando $d > 4$.

2. **Optimización.** Problemas: TSP, Minimum Dominating Set (MDS) y MaxCut (HamLib) (Sawaya et al., 2024; PennyLane, n.d.f,n). Solvers: fuerza bruta, greedy, ILP (Gurobi Optimization, n.d.), QUBO+SA frente a QAOA y VQE (Farhi et al., 2014; Peruzzo et al., 2014; Lubinski et al., 2024). Métricas: coste/corte, gap, éxito y tiempo.

3. **Representación.** Embedding / feature map en QML (PennyLane, n.d.d,n); QUBO–Ising en optimización (Lucas, 2014).

Iris y Bars and Stripes son datasets convencionales para QML; Ising es dato de sistema con etiqueta de fase (PennyLane, n.d.n,n). No hay datasets de empresa en este corpus. La aportación es el protocolo y la lectura de límites, no un claim de ventaja en este régimen (Bowles et al., 2024).

1.3 Estructura del trabajo

- **Capítulo 2 – Contexto y estado de la técnica:** era NISQ, QML, QAOA/VQE y limitaciones de los trabajos previos.
- **Capítulo 3 – Objetivos:** objetivo general y objetivos específicos.
- **Capítulo 4 – Metodología:** entorno, datasets e instancias, representación (feature maps y QUBO/Ising), modelos, solvers y métricas.
- **Capítulo 5 – Resultados y análisis:** resultados por familia, dataset e implementación, y discusión centrada en la representación.
- **Capítulo 6 – Conclusiones:** respuesta a la pregunta y cumplimiento de objetivos.
- **Capítulo 7 – Trabajo futuro:** líneas que extienden el protocolo.



Figura 1.1: Estructura del TFM.

Fuente: elaboración propia.

2. Contexto y estado de la técnica

Este capítulo presenta los fundamentos conceptuales y el estado actual de la técnica en computación cuántica aplicada a la ciencia de datos. Se revisan los trabajos previos más relevantes, se identifican las limitaciones de los enfoques existentes y se sitúa la contribución de este trabajo en el contexto académico y profesional actual.

2.1 Antecedentes históricos

El origen formal de la computación cuántica como disciplina se remonta a los trabajos de Feynman (1982), quien propuso el uso de sistemas cuánticos para simular la naturaleza de forma eficiente, argumentando que un computador clásico no podía simular un sistema cuántico sin un coste exponencial. Poco después, Deutsch (1985) formalizó el concepto de computador cuántico universal y demostró la existencia de problemas que un computador cuántico podría resolver de forma más eficiente que cualquier computador clásico.

El interés por la computación cuántica se intensificó con los algoritmos de Shor (1994) y Grover (1996), que demostraron aceleraciones exponenciales y cuadráticas respectivamente frente a los mejores algoritmos clásicos conocidos para factorización de enteros y búsqueda no estructurada. El primero descansa en la transformada cuántica de Fourier y el segundo en la amplificación de amplitud (PennyLane, n.d.k,n). Estos resultados establecieron la viabilidad teórica de la ventaja cuántica y sentaron las bases para el desarrollo de hardware cuántico experimental. Los fundamentos formales del modelo de computación cuántica, incluyendo los postulados de la mecánica cuántica, las puertas cuánticas y los circuitos cuánticos, se encuentran desarrollados en detalle en Nielsen & Chuang (2010); CERN & Atos (2021) y Quantum Zeitgeist (n.d.b) ofrecen introducciones más aplicadas.

En paralelo, el campo del aprendizaje automático experimentó un crecimiento extraordinario impulsado por el aumento de la capacidad computacional y la disponibilidad de grandes volúmenes de datos (Jordan & Mitchell, 2015). La confluencia de ambas disciplinas dio lugar al campo del aprendizaje automático cuántico (*Quantum Machine Learning*, QML), cuyo objetivo es explorar si las capacidades de procesamiento cuántico pueden ofrecer ventajas en tareas de aprendizaje estadístico (Biamonte et al., 2017). La relación también se da en sentido inverso: el aprendizaje automático se usa para caracterizar estados y controlar dispositivos cuánticos (Fastovets et al., 2019).

2.2 Computación cuántica en la era NISQ

El estado actual del hardware cuántico se enmarca en lo que Preskill (2018) denominó la era NISQ (*Noisy Intermediate-Scale Quantum*), caracterizada por dispositivos con entre 50 y varios cientos de qubits físicos, afectados por errores de decoherencia y ruido cuántico, y sin capacidad para implementar corrección de errores cuánticos a gran escala. Esta realidad impone restricciones fundamentales sobre la profundidad de los circuitos ejecutables y la fiabilidad de los resultados obtenidos.

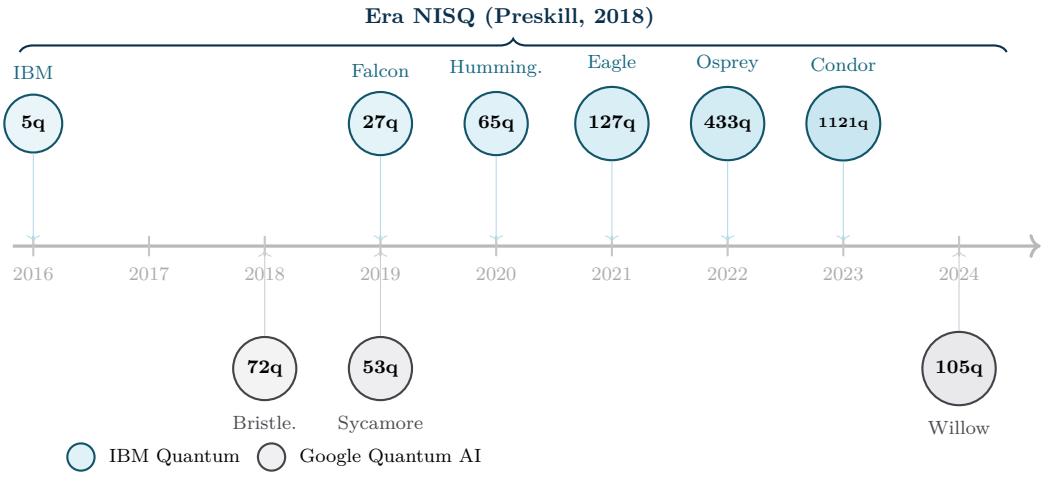


Figura 2.1: Evolución del hardware cuántico en la era NISQ (2016–2024). Los nodos superiores (azul) corresponden a procesadores IBM Quantum; los inferiores (violeta) a Google Quantum AI. Se indica el año de presentación de cada procesador; los recuentos de qubits de IBM proceden de sus anuncios públicos. Los números indican el recuento de qubits físicos disponibles. Arute et al. (2019) presentaron con Sycamore (2019) un experimento de muestreo de circuitos aleatorios que declararon fuera del alcance de la simulación clásica de entonces; Pan et al. (2022) lo replicaron después con algoritmos clásicos mejorados. Willow (2024) se asocia al trabajo de Google Quantum AI (2025) sobre corrección de errores por debajo del umbral del código de superficie.
Fuente: elaboración propia.

En respuesta a estas limitaciones, se han desarrollado los algoritmos variacionales cuánticos (*Variational Quantum Algorithms*, VQA), que combinan un circuito cuántico parametrizado con un optimizador clásico en un bucle híbrido (Cerezo et al., 2021; Quantum Zeitgeist, n.d.a). El diseño del ansatz condiciona su expresividad y su capacidad de entrelazamiento (Sim et al., 2019), y la recarga repetida de los datos en el circuito permite

construir clasificadores universales con pocos qubits (Pérez-Salinas et al., 2020). Entre los más destacados se encuentran el Quantum Approximate Optimization Algorithm (QAOA) (Farhi et al., 2014), diseñado para problemas de optimización combinatoria, y el Variational Quantum Eigensolver (VQE) (Peruzzo et al., 2014), orientado a la simulación de sistemas moleculares. Estos algoritmos son actualmente los candidatos más prometedores para demostrar ventaja cuántica práctica en hardware NISQ.

No obstante, la comunidad científica mantiene un debate activo sobre si los algoritmos NISQ pueden alcanzar ventaja cuántica genuina frente a los mejores algoritmos clásicos. McClean et al. (2018) mostraron que, en circuitos parametrizados aleatorios, la probabilidad de que el gradiente sea distinto de cero con precisión fija es exponencialmente pequeña en el número de qubits (*barren plateaus*), lo que dificulta el entrenamiento de los circuitos variacionales a medida que el número de qubits crece. Cerezo et al. (2025) discuten además si los modelos libres de *barren plateaus* pueden simularse eficientemente de forma clásica, lo que cuestiona qué ventaja pueden ofrecer.

2.3 Aprendizaje automático cuántico

El aprendizaje automático cuántico ha sido objeto de una revisión por parte de Biamonte et al. (2017), quienes identificaron cuatro categorías principales de algoritmos QML según la naturaleza del procesador (clásico o cuántico) y de los datos (clásicos o cuánticos). Para el contexto de este trabajo, el caso más relevante es el de datos clásicos procesados mediante algoritmos cuánticos (CQ en la nomenclatura de los autores: datos clásicos, procesador cuántico).

Los métodos basados en kernels cuánticos (*Quantum Kernel Methods*) han recibido especial atención. Schuld & Killoran (2019) propusieron interpretar los circuitos cuánticos como mapas de características (*feature maps*) hacia espacios de Hilbert de alta dimensión, y Havlíček et al. (2019) implementaron experimentalmente el estimador de kernel cuántico en un procesador superconductor, argumentando que ciertos mapas son difíciles de simular clásicamente. Schuld (2021) mostró después que los modelos cuánticos supervisados basados en circuitos parametrizados son, formalmente, métodos de kernel. Sin embargo, Huang et al. (2021) establecieron condiciones teóricas precisas bajo las cuales los kernels cuánticos pueden ofrecer una ventaja de aprendizaje sobre los kernels clásicos, concluyendo que dicha ventaja depende críticamente de la estructura geométrica de los datos de

entrada. Liu et al. (2021) probaron una aceleración rigurosa en aprendizaje supervisado, pero sobre un problema construido a partir del logaritmo discreto; el grado de acuerdo entre un kernel y las etiquetas puede medirse con el alineamiento centrado de Cortes et al. (2012).

En el contexto de la clasificación supervisada con datos reales, Haug et al. (2023) propusieron el uso de medidas aleatorizadas para escalar los métodos de kernel cuántico a conjuntos de datos de gran tamaño, abordando uno de los principales cuellos de botella prácticos de estos métodos: el coste de estimar el kernel crece cuadráticamente con el tamaño del conjunto de datos. Su método reduce el tiempo de cómputo cuántico a escala lineal con dicho tamaño, aunque en general escala de forma exponencial con el número de qubits, y lo demostraron clasificando imágenes en un procesador cuántico de IBM.

2.4 Optimización cuántica aplicada

El QAOA, propuesto originalmente por Farhi et al. (2014), es un algoritmo híbrido cuántico-clásico que aproxima la solución de problemas de optimización combinatoria formulados como problemas de optimización cuadrática binaria sin restricciones (QUBO). Muchos de esos problemas, como el TSP o el conjunto dominante mínimo, son NP-difíciles (Garey & Johnson, 1979), y el annealing simulado, uno de sus heurísticos clásicos habituales, tiene una versión cuántica con aceleración cuadrática respecto del gap espectral del proceso clásico (Somma et al., 2008). Su aplicabilidad se extiende a problemas de relevancia empresarial como la optimización de rutas (Lucas, 2014), la asignación de recursos, la segmentación de mercado y la optimización de carteras financieras.

Zhou et al. (2020) estudiaron numéricamente el QAOA sobre MaxCut en grafos regulares, hasta $p \leq 50$ y $N \leq 22$ vértices, y propusieron estrategias heurísticas de inicialización de parámetros cuyo coste crece de forma polinómica en p; observaron además que la calidad de la solución mejora al aumentar la profundidad. No obstante, la ventaja cuántica del QAOA frente a los mejores algoritmos clásicos heurísticos sigue siendo una cuestión abierta para instancias de gran escala (Farhi et al., 2014).

Desde una perspectiva aplicada, Vikstål et al. (2020) simularon el QAOA sobre instancias del problema de asignación de aeronaves a vuelos (*tail-assignment*) derivadas de datos reales de una aerolínea, reducidas a 8, 15 y 25 qubits. Encontraron que repetidas ejecuciones identifican la solución factible con probabilidad cercana a uno, que los paráme-

tros variacionales siguen patrones que permiten interpolarlos entre profundidades, y que la probabilidad de éxito por medición depende de la conectividad del grafo del problema. Estos resultados subrayan la importancia de realizar evaluaciones experimentales sobre datos reales más allá de las pruebas de concepto teóricas.

2.5 Análisis crítico de los trabajos previos

La revisión de la literatura permite identificar limitaciones recurrentes en los estudios existentes que este trabajo aborda:

- **Comparaciones asimétricas**: muchos trabajos evalúan el modelo cuántico de forma aislada o frente a baselines clásicos poco ajustados, con preprocesos distintos para cada rama (Bowles et al., 2024). Los benchmarks sistemáticos con el mismo protocolo muestran que los modelos clásicos de referencia suelen igualar o superar a los cuánticos en datasets pequeños (Bowles et al., 2024; XanaduAI, n.d.; Wang, n.d.).
- **Afirmaciones de ventaja sin control**: en la divulgación circulan resultados de superioridad cuántica en optimización que no detallan el solver clásico de referencia ni el tamaño de las instancias (Vocal Media, n.d.). El teorema de *no free lunch* recuerda que ningún optimizador domina sobre todas las clases de problemas (Wolpert & Macready, 1997), de modo que la comparación solo tiene sentido instancia a instancia.
- **Foco en la viabilidad teórica más que en la aplicabilidad práctica**: los análisis de ventaja cuántica suelen formularse en términos asintóticos o bajo supuestos ideales de hardware, sin considerar las restricciones reales de los dispositivos NISQ (Preskill, 2018).
- **Representación implícita**: el feature map o la formulación QUBO se tratan a menudo como un detalle de implementación, aunque determinan qué estructura puede explotar el algoritmo (Huang et al., 2021; Lucas, 2014).

Este trabajo responde a estas limitaciones con un mismo protocolo de preparación y medida para modelos clásicos y cuánticos, con baselines clásicos de referencia y con la representación como eje explícito de análisis.

3. Objetivos

El estado de la técnica del capítulo anterior sitúa la brecha entre promesas teóricas del QML / optimización cuántico-inspirada y lo medible en régimen NISQ (Preskill, 2018; Bowles et al., 2024). Este capítulo fija el objetivo general y los específicos del estudio.

3.1 Objetivo general

El objetivo general de este trabajo es **evaluar de forma empírica y sistemática** si los algoritmos cuánticos disponibles en la era NISQ ofrecen una **ventaja real y medible** frente a los algoritmos clásicos consolidados en problemas representativos de la ciencia de datos, con el fin de determinar las condiciones bajo las cuales la computación cuántica puede aportar valor práctico en este dominio (Preskill, 2018; Biamonte et al., 2017).

3.2 Objetivos específicos

1. **Definir un marco conceptual** que organice el estudio en torno a **clasificación**, **optimización** y **representación** (feature map / QUBO–Ising) (Lloyd et al., 2020; PennyLane, n.d.c; Parzygnat et al., 2025).

2. **Diseñar e implementar pipelines de clasificación** comparables entre modelos clásicos y cuánticos bajo un **protocolo justo** de partición, escalado y reducción dimensional cuando el presupuesto de qubits lo exija (Schuld & Killoran, 2019; Havlíček et al., 2019; Bowles et al., 2024).

3. **Diseñar e implementar pipelines de optimización combinatoria** comparables entre solvers clásicos y cuántico-inspirados (incluidos QAOA y VQE) sobre **formulaciones QUBO/Ising compartidas** (Farhi et al., 2014; Peruzzo et al., 2014; Lucas, 2014; Lubinski et al., 2024).

4. **Evaluar calidad de la solución y esfuerzo computacional** con métricas comunes, de forma **reproducible** (Bowles et al., 2024; Cerezo et al., 2021).

5. **Interpretar los resultados** en el régimen NISQ de pocos qubits y simulación clásica, **delimitando qué se puede y no se puede afirmar** sobre ventaja cuántica (Huang et al., 2021; Thanasilp et al., 2024; McClean et al., 2018; Preskill, 2018).

4. Metodología y desarrollo experimental

Este capítulo describe cómo se ejecutaron los experimentos del TFM: entorno software, diseño experimental, datasets, preparación, representación en el espacio de Hilbert (o QUBO/Ising), modelos/solvers, y el protocolo de medición de calidad y latencia (Bowles et al., 2024).

4.1 Entorno tecnológico

Capa	**Stack**	**Versión**
Lenguaje	Python	3
Notebooks	Jupyter (`ipykernel`, `nbformat`)	—
Numérico / datos	NumPy, pandas, h5py	—
ML clásico	scikit-learn (SVM, Random Forest, PCA, métricas, splits); XGBoost (Chen & Guestrin, 2016)	1.9; 3.1
QML / circuitos	PennyLane (`default.qubit`, `lightning.qubit`) (PennyLane, n.d.m); Qiskit y `qiskit-machine-learning` (kernel de fidelidad) (Qiskit contributors, 2026)	0.45; 2.6
QNN híbrida	PyTorch (`TorchLayer` + cabeza lineal)	—
Optimización	PuLP + CBC (ILP); NetworkX (grafos); annealing simulado propio sobre QUBO	2.9; 3.7
Visualización	matplotlib y utilidades propias de storytelling	—

Cuadro 4.1: Entorno software del estudio. Las versiones corresponden a las dependencias fijadas del repositorio.

Todo el cómputo cuántico de este TFM es **simulación** de estado (sin hardware ruidoso) (Preskill, 2018).

4.2 Diseño experimental

El estudio se organiza en tres ejes. Dos son familias de casos de uso; el tercero, la representación, es el paso que lleva el dato o el problema al espacio donde opera el método

cuántico, y está presente en ambas (Lloyd et al., 2020; PennyLane, n.d.c; Parzygnat et al., 2025).

Eje	**Pregunta**	**Casos del estudio**
Clasificación (*qué*)	¿Qué decisión tomar sobre el dato?	Iris, Bars and Stripes, Ising
Optimización (*cómo*)	¿Cuál es la mejor configuración?	TSP, MDS, MaxCut
Representación	¿En qué espacio se resuelve?	Feature map; QUBO–Ising

Cuadro 4.2: Ejes del estudio. La representación no es una tercera familia de datos: es la transformación que comparten clasificación y optimización.

En clasificación, la representación es el feature map $\mathbf{x} \mapsto |\phi(\mathbf{x})\rangle$ (Sección 4.3.3). En optimización, es la reformulación del problema combinatorio como QUBO o Hamiltoniano de Ising (Sección 4.4.2) (Lucas, 2014). Otras familias, como regresión, clustering, series temporales, detección de anomalías, aprendizaje por refuerzo, modelos generativos o simulación, admiten el mismo planteamiento, pero quedan fuera del diseño experimental.

Sobre esos ejes se fijan dos familias con la misma lógica de comparación justa (Bowles et al., 2024; XanaduAI, n.d.):

1. **Clasificación:** mismos splits, mismo escalado y, cuando $d > 4$, misma PCA a 4 componentes para *todos* los modelos (clásicos y cuánticos).

2. **Optimización:** misma instancia por problema; mismo óptimo de referencia; solvers clásicos y cuántico-inspirados evaluados con las mismas métricas de calidad y tiempo.

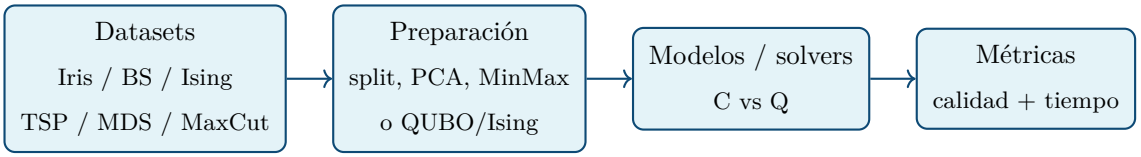


Figura 4.1: Pipeline experimental común: dato → preparación → modelo/solver → métricas agregadas.
Fuente: elaboración propia.

4.3 Clasificación

4.3.1 Datasets

Los tres datasets difieren en el origen del dato. Iris y Bars and Stripes son datasets *convencionales*: el dato es clásico y cualquier estructura cuántica la introduce el feature map. Ising es dato *de sistema*: cada muestra procede del estado fundamental de un Hamiltoniano y la etiqueta es su fase (PennyLane, n.d.n,n,n). La diferencia condiciona qué representación tiene sentido.

Iris. Benchmark clásico de 150 muestras, 4 características continuas y 3 clases (scikit-learn developers, n.d.). Se carga con `sklearn.datasets.load_iris`. Con $d = 4$ cabe en 4 qubits **sin PCA**. La Figura 4.2 es el dato antes de cualquier clasificador: las cuatro variables proyectadas solo para verlas. Una clase queda separada; las otras dos se solapan.

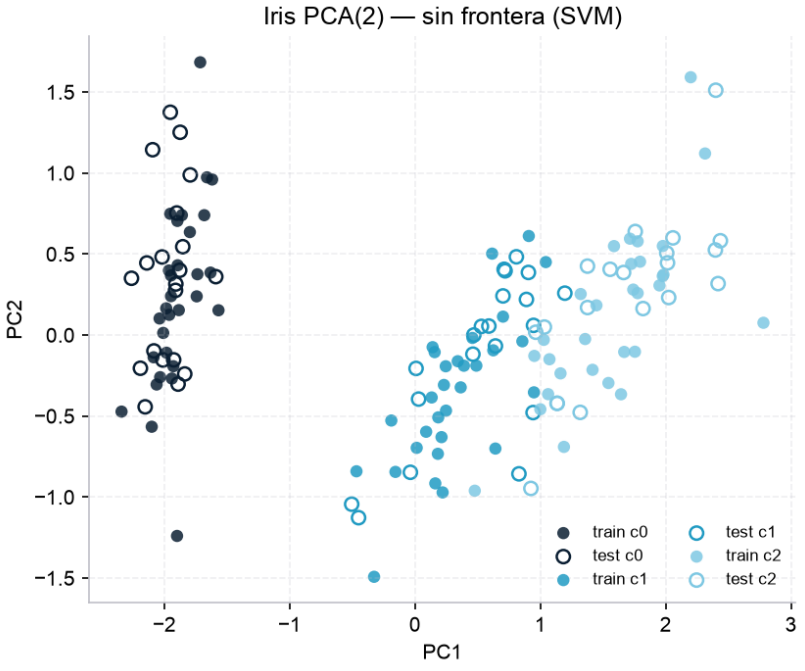


Figura 4.2: Iris antes de los clasificadores. Proyección visual de las cuatro variables originales; el modelo no usa esta PCA. Círculos huecos: test del protocolo.
Fuente: elaboración propia.

Bars and Stripes (BS). Patrones binarios 4×4 del catálogo PennyLane (`bars-and-stripes`) (PennyLane, n.d.a,n). Es un dataset *convencional* etiquetado para QML, no un "quantum dataset" de sistema (PennyLane, n.d.n). Tras aplanar, $d = 16$; se comprime con PCA a 4. La Figura 4.3 muestra el patrón antes de ese aplanado.

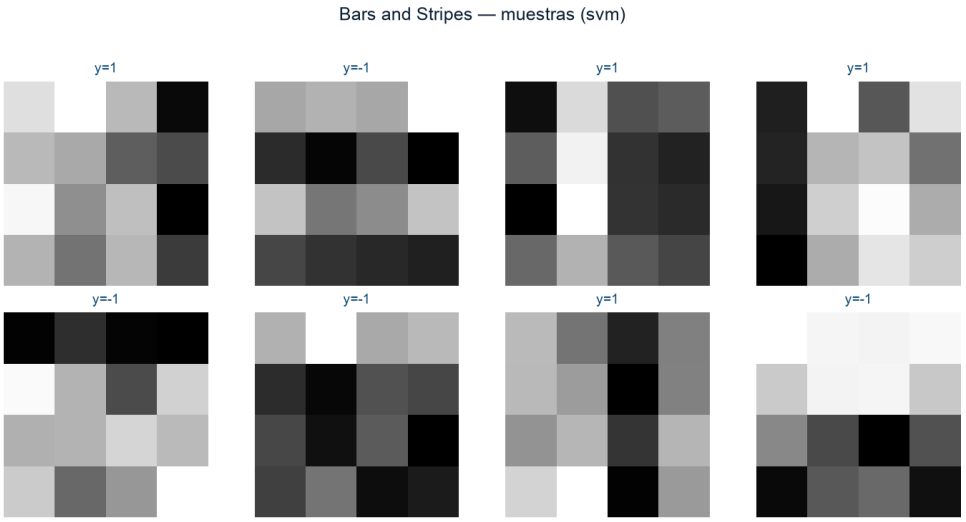


Figura 4.3: Bars and Stripes antes de aplanar y antes de cualquier clasificador. Cada panel es un patrón 4×4; $y \in \{+1, -1\}$ es la clase del catálogo.

Fuente: elaboración propia.

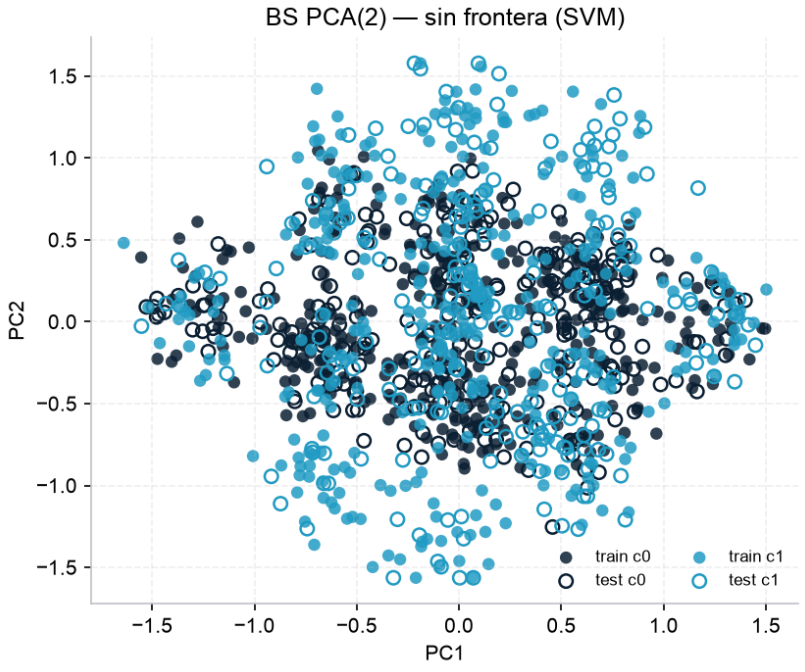


Figura 4.4: Bars and Stripes tras la reducción a componentes y antes de cualquier clasificador. Las dos clases comparten el mismo plano. Círculos huecos: test.

Fuente: elaboración propia.

Ising (fases). Datos PennyLane `qspin` / Ising (PennyLane, n.d.h): amplitudes del ground state (parte real e imaginaria) de cadenas 1D de cuatro espines (Figura 4.5). La etiqueta de fase se deriva del campo transversal, $y = \mathbf{1}[h > |J|]$; h **no** entra como feature (evitar fuga de etiqueta). Tras el aplanado, $d > 4$; se aplica PCA a 4. El split estratifica también por h en QNN/VQC.

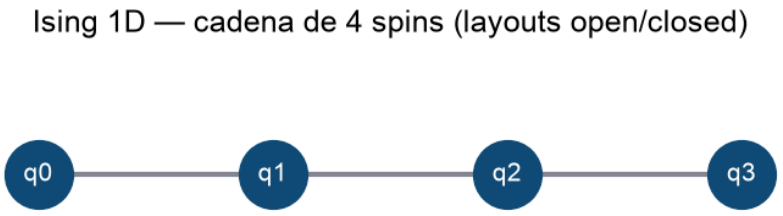


Figura 4.5: Sistema de Ising: cadena 1D de cuatro espines con acoplamiento J entre vecinos y campo transversal h, en layout abierto o cerrado. Las features son las amplitudes del estado fundamental de este Hamiltoniano para cada h.
Fuente: elaboración propia.

4.3.2 Preparación compartida

Protocolo fijo en los tres datasets:

1. `train_test_split` estratificado, `test_size`=0.4, `random_state`=123.
2. Si $d > 4$: `PCA(n_components=4, random_state=123)` ajustada solo en train y aplicada a test.
3. `MinMaxScaler(feature_range=(0, π))` ajustado en train (rango compatible con angle / ZZ encoding) (PennyLane, n.d.c).

La PCA, cuando existe, se aplica a clásicos y cuánticos por igual: se compara el modelo, no un preproceso distinto (Bowles et al., 2024).

Las Figuras 4.2 y 4.4 son esa geometría antes de ajustar ningún modelo. En Iris la proyección es solo visual ($d = 4$); en Bars and Stripes el clasificador recibe cuatro componentes. En Ising la proyección no aporta lectura física: el dato se entiende desde el sistema de la Figura 4.5.

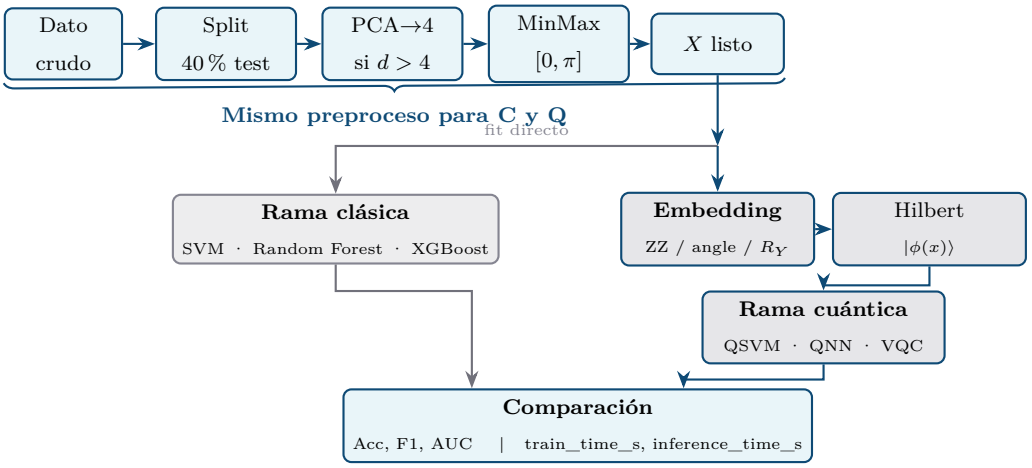


Figura 4.6: Preparación compartida y bifurcación C/Q. Tras split, PCA (si $d > 4$) y MinMax$[0, \pi]$, el mismo X alimenta baselines clásicos; en la rama cuántica pasa antes por el embedding al espacio de Hilbert. Ambas ramas se evalúan con las mismas métricas de calidad y latencia.

Fuente: elaboración propia.

4.3.3 Representación en el espacio de Hilbert

El feature map es un mapeo $\phi\colon \mathcal{X} \to \mathcal{H}$ que toma los datos en su espacio original $\mathcal{X} \subseteq \mathbb{R}^d$ y los transforma a estados en el espacio de Hilbert $\mathcal{H} = (\mathbb{C}^2)^{\otimes n} \simeq \mathbb{C}^{2^n}\colon \mathbf{x} \mapsto |\phi(\mathbf{x})\rangle$ (Lloyd et al., 2020; PennyLane, n.d.d; Schuld & Killoran, 2019; Parate, 2025). Tras PCA (si $d > n$) y, cuando el mapa es angular o ZZ, MinMax$[0, \pi]$, cada muestra se prepara con un circuito unitario $U_\phi(\mathbf{x})$ tal que $|\phi(\mathbf{x})\rangle = U_\phi(\mathbf{x})|0\rangle^{\otimes n}$. Sin ese embedding el dato clásico no es un estado cuántico; la utilidad de ϕ depende de la alineación entre la geometría inducida en $\mathcal{H}$ y la estructura del generador de etiquetas (Huang et al., 2021; PennyLane, n.d.c; Parzygnat et al., 2025). Con presupuesto $n = 4$, $\mathcal{H} \simeq \mathbb{C}^{16}$. Se consideran cuatro embeddings:

- **Basis encoding.** Codifica cadenas binarias (o categorías discretizadas) en la base computacional: para $\mathbf{b} \in \{0,1\}^n$, $|\phi(\mathbf{b})\rangle = |b_1 \cdots b_n\rangle = \bigotimes_i X^{b_i}|0\rangle$ (Parate, 2025; PennyLane, n.d.d,n). Los estados son ortogonales; una medida en Z recupera $\mathbf{b}$ con certeza. Escala como $\lceil \log_2 C \rceil$ qubits para C categorías, pero no es natural para features continuas sin cuantización previa.

- **Mapa ZZ (`ZZFeatureMap`).** Preparación producto $H^{\otimes n}$ seguida de rotaciones $R_Z(x_i)$ y puertas de fase entrelazadas $e^{-i\,\theta_{ij} Z_i Z_j}$ con $\theta_{ij} \propto (\pi - x_i)(\pi - x_j)$

(`entanglement=full`, alineado con el mapa de Qiskit) (Havlíček et al., 2019). El producto interno en $\mathcal{H}$ define el kernel de fidelidad $K(\mathbf{x}, \mathbf{x}') = |\langle\phi(\mathbf{x})|\phi(\mathbf{x}')\rangle|^2$.

- **Angle embedding.** `AngleEmbedding` aplica rotaciones locales $R_\sigma(x_i)$ ($\sigma \in \{X, Y, Z\}$) sobre el qubit i, de modo que $|\phi(\mathbf{x})\rangle = \bigotimes_{i=1}^{n} R_\sigma(x_i)|0\rangle$ (PennyLane, n.d.d,n; Parate, 2025). Codificación angular separable: no introduce entrelazamiento en la preparación del estado.

- **Preparación $R_Y(x_i)$.** Caso particular de angle embedding con $\sigma = Y$: $U_\phi(\mathbf{x}) = \bigotimes_i R_Y(x_i)$, un parámetro de rotación por dimensión de feature (PennyLane, n.d.m; Cerezo et al., 2021).

Las capas parametrizadas posteriores (`BasicEntanglerLayers`, `Rot`+CNOT), el estimador de K y la cabeza de clasificación no forman parte de ϕ: pertenecen al modelo. En los experimentos de clasificación, tras el protocolo compartido (PCA + MinMax$[0, \pi]$), se emplean mapa ZZ, angle embedding y R_Y; el basis encoding queda como alternativa para datos ya binarios o categóricos (p. ej. Bars and Stripes antes de la compresión) (Parate, 2025).

4.3.4 Modelos

Familia	Modelo	Implementación
Clásica	SVM (RBF) (Cortes & Vapnik, 1995)	`sklearn.svm.SVC`
Clásica	Random Forest (Breiman, 2001)	`sklearn.ensemble`
Clásica	XGBoost (Chen & Guestrin, 2016)	`XGBClassifier`
Cuántica	QSVM (Havlíček et al., 2019)	mapa ZZ + kernel fidelidad + `SVC`
Cuántica	QNN (PennyLane, n.d.l)	angle embedding + `BasicEntanglerLayers` + cabeza PyTorch
Cuántica	VQC (PennyLane, n.d.m)	prep. $R_Y(x_i)$ + capas `Rot`+CNOT (Nesterov)

Los seis clasificadores se evalúan sobre las mismas particiones y el mismo $\mathbf{x}$ post-preproceso. La rama clásica opera directamente en $\mathbb{R}^n$. En la rama cuántica, cada modelo fija un U_ϕ y una regla de decisión sobre $|\phi(\mathbf{x})\rangle$: el QSVM estima la matriz Gram de fidelidad

$K(\mathbf{x}, \mathbf{x}') = |\langle\phi(\mathbf{x})|\phi(\mathbf{x}')\rangle|^2$ y la entrega a un SVM con kernel precomputado; la QNN y el VQC componen U_ϕ con un ansatz variacional y clasifican a partir de valores esperados $\langle Z_i\rangle$ (con cabeza lineal entrenable en el caso híbrido QNN) (Havlíček et al., 2019; PennyLane, n.d.m; Cerezo et al., 2021). Así, ϕ define la representación; el modelo define cómo se lee esa representación.

4.3.5 Métricas de clasificación y latencia

Tras el ajuste, la calidad se mide en el conjunto de test; el conjunto de train entra solo en el gap de generalización. La Tabla 4.3 fija las definiciones. F1 macro es la métrica de ordenación entre modelos.

Métrica	**Definición**
Accuracy	Fracción de etiquetas correctas, $\text{Acc} = \frac{1}{N}\sum_i \mathbf{1}[\hat{y}_i = y_i]$.
Precision, recall	Medias macro sobre las clases.
F1	Media armónica macro de precision y recall.
AUC	Área bajo la curva ROC cuando el modelo produce scores.
Gap de generalización	$\text{Acc}_{\text{train}} - \text{Acc}_{\text{test}}$.
Latencia	Wall-clock de entrenamiento (`fit`) y de inferencia sobre el test.

Cuadro 4.3: Métricas de calidad y latencia en clasificación.

4.4 Optimización

4.4.1 Problemas e instancias

TSP restringido. TSP y MDS son NP-difíciles en su versión general (Garey & Johnson, 1979); aquí se usan instancias pequeñas con óptimo verificable. Grafo completo pequeño sobre 5 nodos con distancias fijas; origen fijado (Figura 4.7). Óptimo por enumeración (fuerza bruta). Formulación QUBO reducida: variables binarias sobre rutas candidatas factibles (una sola familia de selección), compatible con SA/QAOA/VQE (Lucas, 2014).

Minimum Dominating Set (MDS). Grafo social de 6 vértices y 6 aristas (Figura 4.8). Objetivo: mínimo conjunto dominante. ILP de cobertura con PuLP/CBC; QUBO para SA/QAOA/VQE.

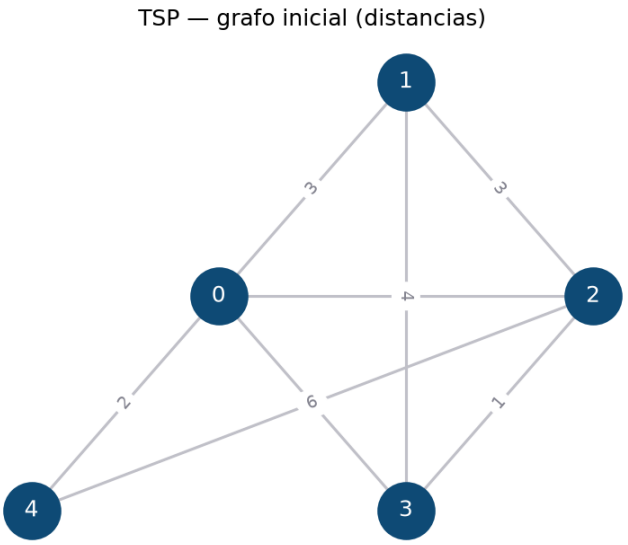


Figura 4.7: Instancia de TSP antes de cualquier solver. Cinco nodos y las distancias de las aristas.

Fuente: elaboración propia.

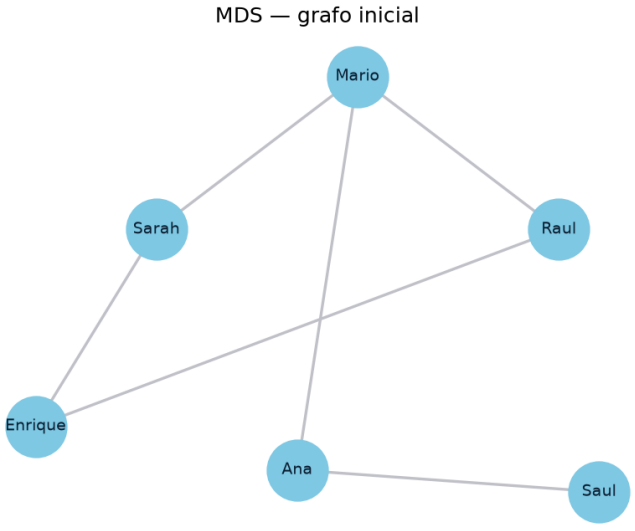


Figura 4.8: Instancia de MDS antes de cualquier solver. Seis vértices y seis aristas.

Fuente: elaboración propia.

MaxCut (HamLib). Instancia HamLib (`hamlib_maxcut.json`: $n = 6$, 12 aristas, corte óptimo 8, id `circ_offsets-1-2`; Figura 4.9) (Sawaya et al., 2024; PennyLane, n.d.f). Circuito QAOA al estilo del demo PennyLane MaxCut (PennyLane, n.d.j; Farhi et al., 2014).

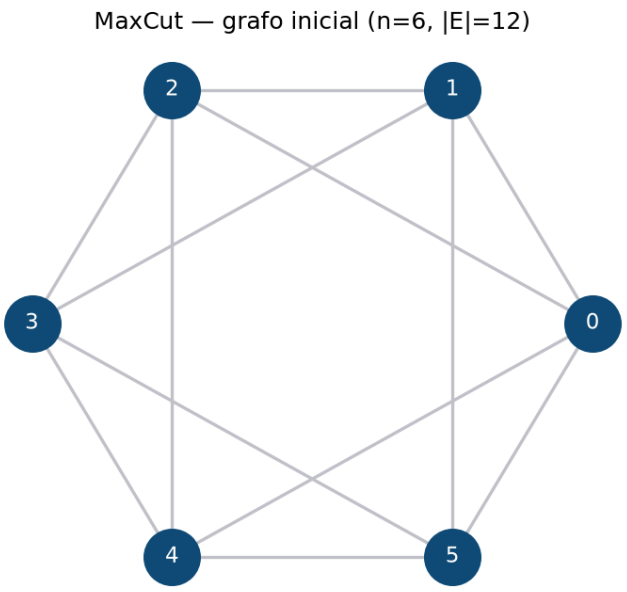


Figura 4.9: Instancia de MaxCut antes de cualquier solver. Seis vértices y doce aristas; el corte óptimo es 8.
Fuente: elaboración propia.

4.4.2 Preparación: de problema a QUBO/Ising

La reformulación QUBO–Ising es la cara de la representación en optimización: convierte el problema combinatorio en la minimización de una forma cuadrática sobre variables binarias, o de forma equivalente en la energía de un Hamiltoniano de espines H_C (Lucas, 2014). Las tres instancias no plantean la misma exigencia. MaxCut, tomado de HamLib, es el caso canónico de QAOA: el corte se escribe directamente como suma de términos Z_iZ_j y no tiene restricciones (Sawaya et al., 2024; PennyLane, n.d.f,n). TSP y MDS tienen restricciones (visitar cada nodo una vez, dominar todos los vértices), que se introducen como penalizaciones. Ahí la codificación tiene que preservar a la vez el coste y la factibilidad.

La calidad de la representación importa tanto como el solver: un QUBO mal penalizado optimiza otra función (Lucas, 2014; Lubinski et al., 2024).

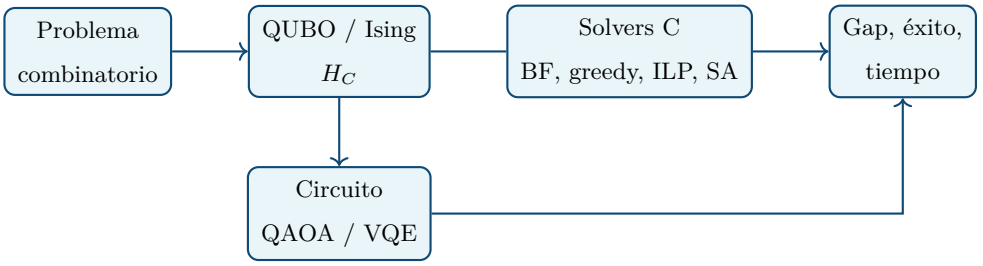


Figura 4.10: Optimización: la representación QUBO/Ising es compartida por SA, QAOA y VQE; BF/greedy/ILP operan sobre la formulación nativa o equivalente.
Fuente: elaboración propia.

4.4.3 Solvers

Familia	**Solver**	**Notas**
Clásica	Fuerza bruta	Óptimo de referencia
Clásica	Greedy	Heurística rápida
Clásica	ILP (PuLP+CBC)	Exacto (Gurobi Optimization, n.d.)
Clásica	QUBO + SA	Annealing simulado propio
Cuánt.-insp.	QAOA	PennyLane, capas $U_C U_B$ (Farhi et al., 2014; PennyLane, n.d.i)
Cuánt.-insp.	VQE	Ansatz + energía del Ising (Peruzzo et al., 2014)

QAOA/VQE se ejecutan en simulación de estado (`default.qubit`); se reportan varias semillas cuando el protocolo lo define.

4.4.4 Métricas de optimización y latencia

La calidad se mide respecto del óptimo conocido $C^\star$ de cada instancia (TSP, MDS, MaxCut). La Tabla 4.4 fija las definiciones. Las series por problema se leen en curvas gap–runtime, success–runtime y heatmaps (Lubinski et al., 2024).

4.5 Comparación clásico vs cuántico

Clásico y cuántico se comparan a lo largo de todo el ciclo de modelado: representación, elección del modelo o del solver, ajuste y lectura del resultado. Cada etapa tiene una lectura cualitativa, que fija la hipótesis del método, y una cuantitativa, que la mide con las métricas de las Tablas 4.3 y 4.4 (Bowles et al., 2024; Lubinski et al., 2024; Yu et al., 2026).

Métrica	**Definición**
Coste / corte	Valor medio y mejor de la función objetivo (longitud, tamaño del dominante o corte).
Gap absoluto	$\lvert C - C^\star \rvert$.
Gap relativo	$\lvert C - C^\star \rvert / \lvert C^\star \rvert$.
Success rate	Fracción de ejecuciones con $C = C^\star$.
Latencia	Tiempo medio wall-clock y, si el solver evalúa un Hamiltoniano, número medio de evaluaciones de energía.

Cuadro 4.4: Métricas de calidad y latencia en optimización. C es el coste o el corte obtenido; $C^\star$ es el óptimo de la instancia.

Etapa	**Cualitativo**	**Cuantitativo**
Representación	En clasificación, $\mathbf{x} \in \mathbb{R}^n$ frente a $\lvert \phi(\mathbf{x}) \rangle$; en optimización, formulación nativa frente a QUBO/Ising.	Dimensión de $\mathcal{H}$ ($n = 4$) y número de variables binarias.
Modelo / solver	Regla de decisión: kernel, árboles, boosting, ansatz variacional, enumeración, ILP o annealing.	Familia (clásica, cuántica o cuántico-inspirada) y criterio de parada.
Ajuste	Mismo split y escalado, o la misma instancia y el mismo $C^\star$.	Wall-clock de entrenamiento u optimización; evaluaciones de energía cuando el solver las cuenta.
Lectura	Generalización del clasificador, o distancia de la solución a $C^\star$, junto con su coste temporal.	Accuracy, F1, AUC, gap de generalización; coste, gap a $C^\star$, success rate y latencia.

Cuadro 4.5: Ciclo de modelado comparado. La columna cualitativa delimita qué es común y qué es propio de cada familia; la cuantitativa aplica las métricas de calidad y latencia.

4.6 Reproducibilidad

- Semillas fijadas en splits y, cuando aplica, en solvers.
- Resolución uniforme del directorio de métricas por caso experimental.
- CSV/JSON y figuras versionados junto al código del experimento.
- Dependencias fijadas en `requirements.txt`.

4.7 Limitaciones del diseño

- El cómputo cuántico es simulación de estado ideal (`default.qubit`, `lightning.qubit`). No hay ejecución en hardware cuántico, así que ruido, conectividad del chip y tiempo de cola quedan fuera del diseño (Preskill, 2018).
- El presupuesto es $n = 4$ qubits y las instancias combinatorias son pequeñas. El alcance es ese régimen de pocas variables (Thanasilp et al., 2024).
- Los hiperparámetros cuánticos quedan fijados por el protocolo, sin barrido exhaustivo.
- En Ising la etiqueta se deriva del campo transversal y el tamaño muestral lo marca el dataset.

5. Resultados y análisis

Este capítulo responde a la pregunta del trabajo con los datos del corpus: bajo un protocolo común y en simulación de pocos qubits, qué hacen los métodos cuánticos y cuántico-inspirados frente a sus referentes clásicos, en calidad y en coste. El Capítulo 4 fijó el dato, la representación, los modelos y las métricas. Aquí se ejecuta esa comparación y se interpreta, de lo particular a lo general.

La presentación sigue las dos familias de casos de uso. En **clasificación** se recorren Iris, Bars and Stripes e Ising; en **optimización**, TSP, MDS y MaxCut. Cada dataset o instancia abre con una tabla con las seis implementaciones y las métricas de las Tablas 4.3 y 4.4. En clasificación son accuracy, F1, AUC, gap de generalización y latencias; en optimización, success rate, coste, gap al óptimo $C^\star$ y tiempo. Debajo, cada implementación tiene su propio apartado.

Cada uno de esos apartados lee la implementación desde tres ángulos. El **cuantitativo** sitúa sus cifras respecto del resto: dónde gana, dónde empata y a qué coste temporal. El **visual** usa la figura que produce el propio método: la frontera de decisión sobre el plano principal en clasificación, o la ruta, el conjunto o el corte devuelto en optimización. Ahí se ve la causa del error o del acierto que la tabla solo resume. El **técnico** relaciona ese comportamiento con la hipótesis del método: el kernel, el bagging o el boosting en los clásicos; el feature map y el ansatz en los cuánticos; la formulación QUBO y el bucle variacional en optimización (Huang et al., 2021; Cerezo et al., 2021; Lucas, 2014).

Cada familia cierra con una lectura cruzada. La pregunta no es solo quién gana una tabla. Importa qué técnica explica cada resultado, cuándo la representación cuántica está alineada con el generador del dato, y si una mejora de calidad compensa el tiempo de simular el circuito (Bowles et al., 2024; Lubinski et al., 2024). Esa lectura deja abiertos tres debates que retoman los capítulos siguientes: si los empates observados son una señal del método o del tamaño del problema; qué cambiaría en hardware real, con ruido y sin simulación de estado; y qué indican estos resultados sobre dónde buscar ventaja más allá de demostraciones pequeñas (Preskill, 2018; Thanasilp et al., 2024).

5.1 Clasificación

La media sobre modelos y sobre Iris, Bars and Stripes e Ising deja a la familia clásica por encima en accuracy, F1 y AUC (Figura 5.1). Esa media aplana tres geometrías distintas. El coste de entrenamiento (Figura 5.2) separa el SVM, que concentra F1 alto y tiempo bajo, del VQC, cuyo F1 medio no compensa decenas de segundos de circuito simulado (Cerezo et al., 2021).

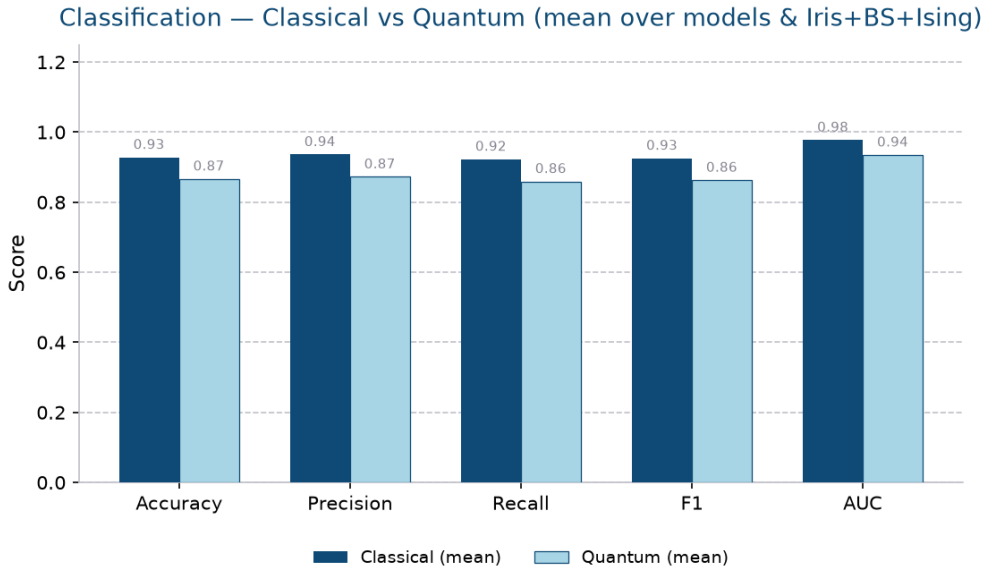


Figura 5.1: Media de la familia clásica y de la cuántica sobre Iris, Bars and Stripes e Ising. El AUC cuántico no incluye el VQC, que no produce scores.
Fuente: elaboración propia.

5.1.1 Iris

Iris entra con $d = 4$, sin PCA en el modelo. La proyección a dos componentes solo sirve para ver la frontera. Setosa queda aislada; las otras dos clases se solapan. La Tabla 5.1 y la Figura 5.3 ordenan los seis clasificadores.

SVM

El kernel RBF corta el plano en tres bandas suaves (Figura 5.4). Setosa queda a un lado; el solape de las otras dos clases deja dos errores de test. F1 0,967, AUC 0,998 y un gap de 0,011 dicen que el margen generaliza. El ajuste cabe en 2 ms. En este dataset el kernel clásico ya está alineado con la geometría: más capacidad no compra accuracy.

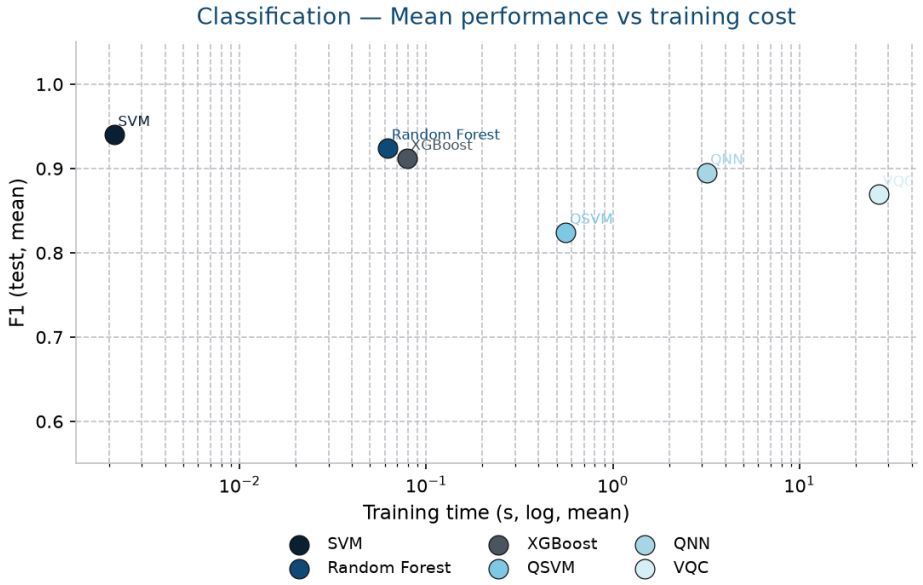


Figura 5.2: F1 medio de test frente al tiempo de entrenamiento (escala logarítmica). El SVM ocupa la esquina de alta calidad y bajo coste; QNN y VQC pagan el bucle variacional. *Fuente: elaboración propia.*

Modelo	**Acc**	**F1**	**AUC**	**Gap**	t_{train} (s)	t_{pred} (s)
SVM (RBF)	0.967	0.967	0.998	0.011	0.002	0.0003
Random Forest	0.950	0.950	0.991	0.050	0.046	0.004
XGBoost	0.950	0.950	0.983	0.050	0.150	0.001
QSVM	0.733	0.730	0.883	0.233	0.072	0.101
QNN	0.933	0.933	0.975	0.022	1.070	0.006
VQC	0.950	0.950	—	−0,006	49.209	0.154

Cuadro 5.1: Iris, conjunto de test. Gap = $\text{Acc}_{\text{train}} - \text{Acc}_{\text{test}}$.

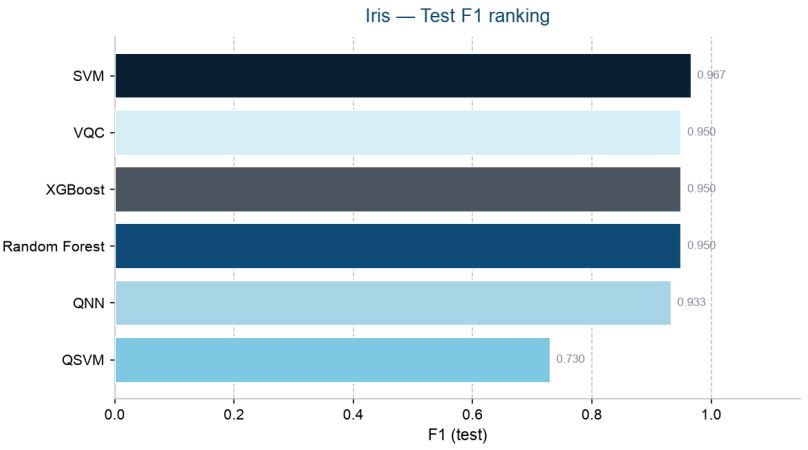


Figura 5.3: Ranking de F1 en Iris. El SVM encabeza la lista; el QSVM es el único que cae de forma clara.

Fuente: elaboración propia.

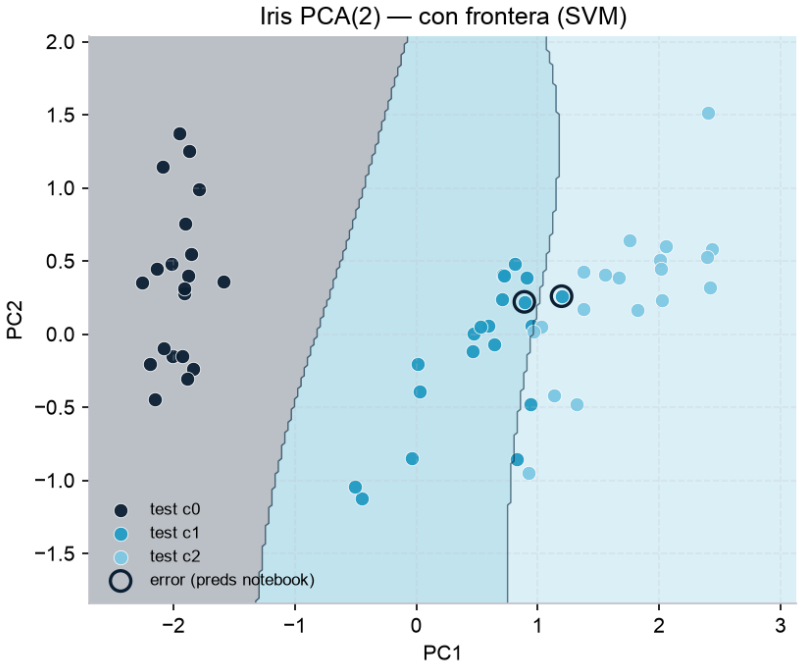


Figura 5.4: Frontera del SVM en las dos primeras componentes de Iris. Los círculos marcan errores de test, concentrados en el solape de las dos clases no separables.

Fuente: elaboración propia.

Random Forest

El bosque iguala el F1 de XGBoost (0,950) con AUC 0,991. El gap sube a 0,050: algunos árboles memorizan el solape. La frontera (Figura 5.5) es una partición por ejes, más quebrada que la del RBF, suficiente para setosa y menos estable en la frontera de las otras dos clases. El coste sigue siendo de centésimas de segundo. Como técnica, el bagging reduce varianza, pero con 150 muestras y tres clases el promedio de árboles no supera un kernel liso.

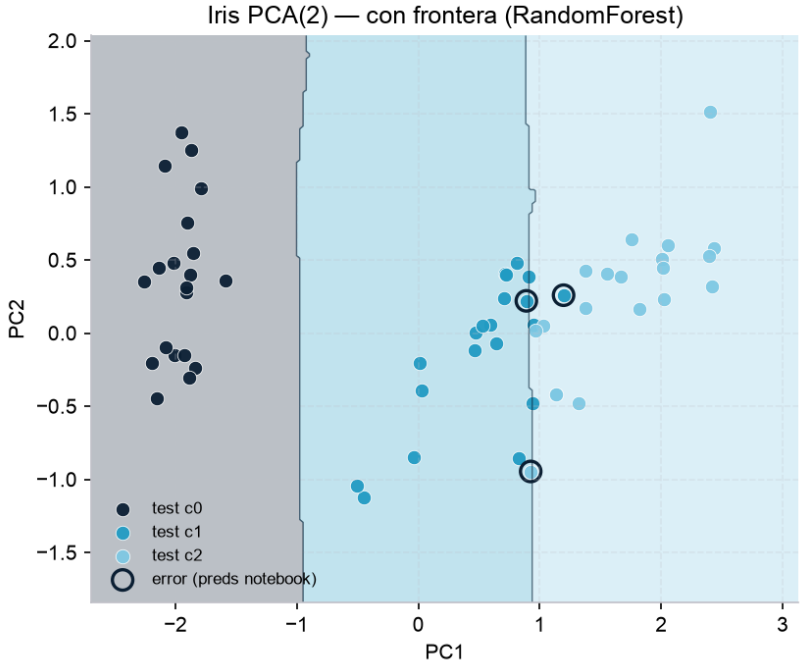


Figura 5.5: Frontera del Random Forest en Iris. La partición es local; el gap de 0,050 aparece donde las dos clases solapadas cambian de hoja.
Fuente: elaboración propia.

XGBoost

Mismas cifras de calidad que el bosque (F1 0,950, gap 0,050) y AUC algo menor (0,983). El entrenamiento, 0,150 s, es el más lento de los clásicos porque el boosting encadena árboles. La Figura 5.6 muestra una frontera por umbrales, del mismo tipo que el bosque. En Iris el gradiente boosting no explota interacciones que el RBF no cubra ya: el dato es pequeño y casi linealmente separable en la primera clase.

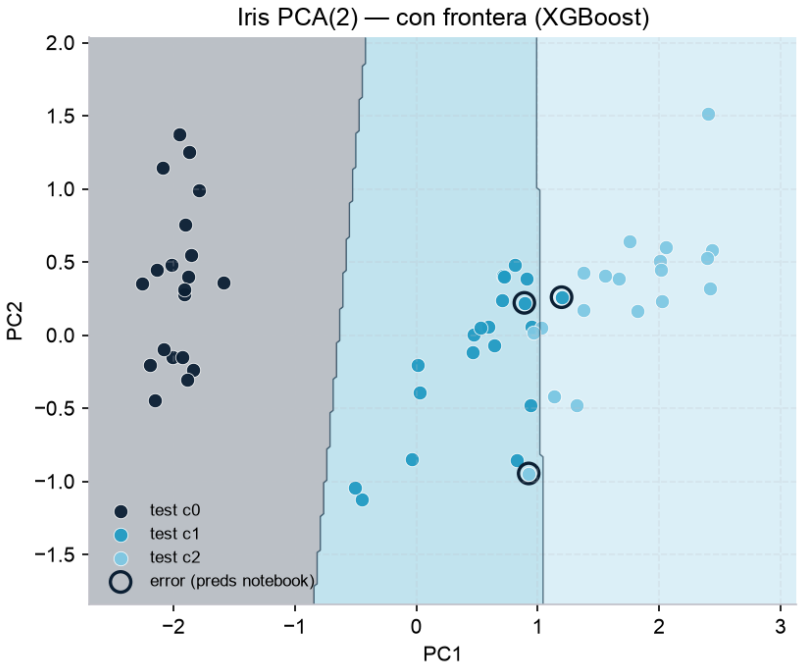


Figura 5.6: Frontera de XGBoost en Iris. La calidad empata con el bosque; el tiempo de ajuste es el mayor de la rama clásica.
Fuente: elaboración propia.

QSVM

El mapa ZZ con kernel de fidelidad es el peor modelo del dataset: F1 0,730, AUC 0,883, gap 0,233. La Figura 5.7 muestra por qué. La frontera en el plano principal está fragmentada: islas de clase dentro de regiones que el RBF deja homogéneas, y errores repartidos también en la clase que el SVM separa de un corte. El producto $Z_i Z_j$ induce un kernel sensible a correlaciones de ángulo, y el generador de etiquetas de Iris no tiene esa forma (Havlíček et al., 2019; Huang et al., 2021). El AUC, todavía alto respecto del accuracy, indica que el orden de los scores es más útil que el umbral duro. La inferencia (0,101 s) cuesta más que el ajuste: cada punto de test exige fidelidades contra el conjunto de soporte, en simulación de estado.

QNN

Angle embedding, capas entrelazadas y una cabeza lineal recuperan F1 0,933 y AUC 0,975, a 0,033 del SVM, con gap 0,022. La cabeza clásica lee $\langle Z_i \rangle$ y puede suavizar una codificación angular que, por sí sola, es un producto de rotaciones. El precio está en el

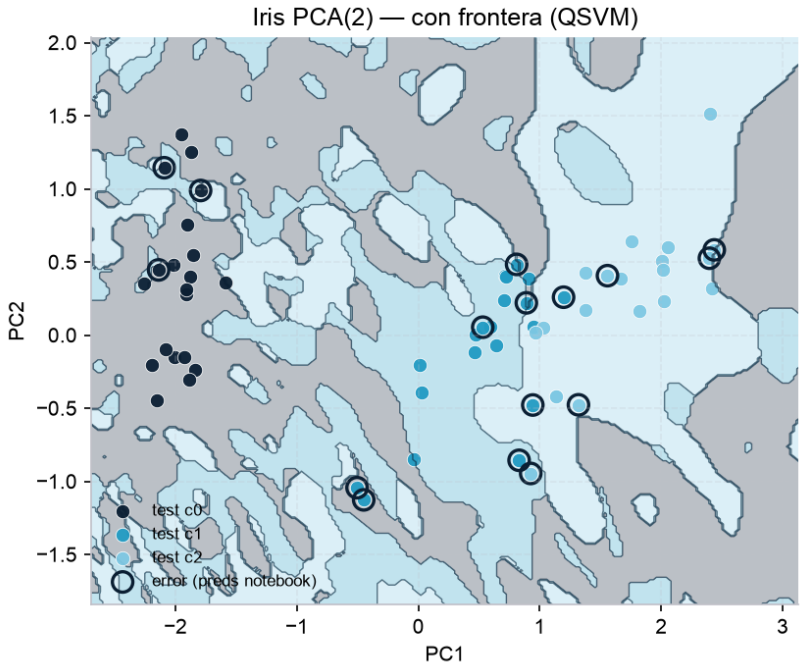


Figura 5.7: Frontera del QSVM (mapa ZZ) en Iris. El kernel fragmenta el plano; los errores no se concentran solo en el solape.
Fuente: elaboración propia.

entrenamiento (1,07 s): cada época evalúa el circuito. La inferencia baja a 0,006 s porque la cabeza ya está ajustada. La Figura 5.8 muestra una frontera más continua que la del QSVM. La técnica híbrida acerca el modelo cuántico al clásico cuando el mapa no está alineado; no lo adelanta.

VQC

Preparación $R_Y(x_i)$ y ansatz `Rot`+CNOT, optimizado con Nesterov, alcanzan el mismo F1 que el bosque y que XGBoost (0,950). El gap es ligeramente negativo (−0,006): el test no queda por debajo del train. La Figura 5.9 es una frontera variacional sobre el mismo plano. Lo que separa al VQC del SVM es el coste: 49,2 s de entrenamiento y 0,154 s de inferencia, porque cada predicción vuelve a ejecutar el circuito. Con $n = 4$ y un dato que un RBF resuelve en milisegundos, el circuito variacional reproduce la calidad clásica intermedia y multiplica el tiempo (Cerezo et al., 2021; Bowles et al., 2024).

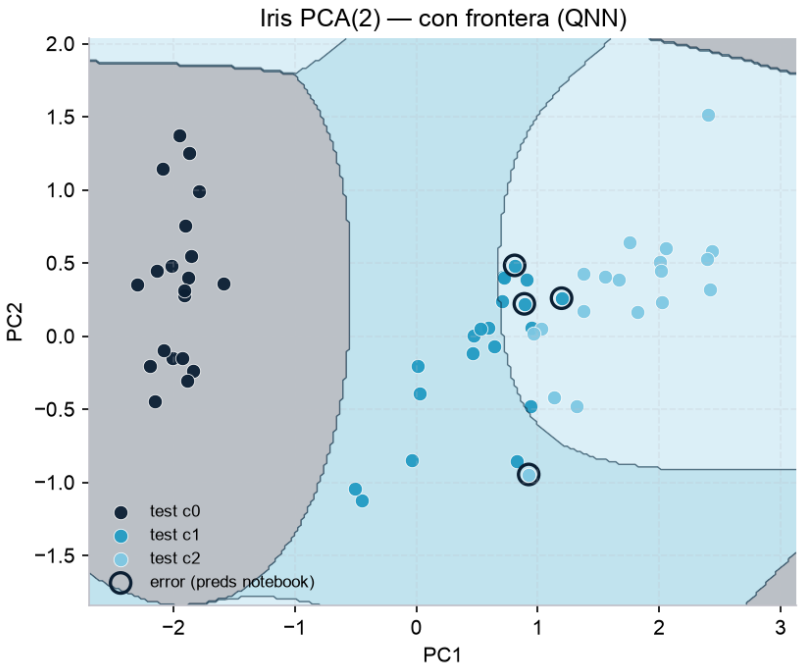


Figura 5.8: Frontera de la QNN en Iris. La cabeza lineal sobre $\langle Z_i \rangle$ produce regiones más compactas que el kernel ZZ.

Fuente: elaboración propia.

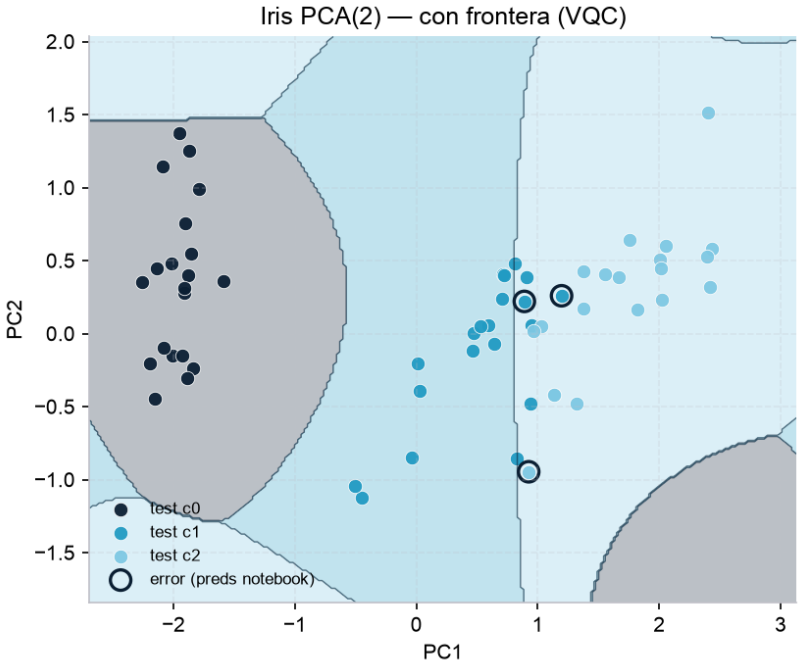


Figura 5.9: Frontera del VQC en Iris. La calidad empata con los árboles; el tiempo de entrenamiento es dos órdenes por encima del SVM.

Fuente: elaboración propia.

5.1.2 Bars and Stripes

El patrón 4×4 se aplana ($d = 16$) y se reduce a cuatro componentes. En el plano de las dos primeras las dos clases se mezclan, así que ninguna frontera suave es suficiente. La Tabla 5.2 es el dataset donde la rama cuántica queda más lejos del mejor clásico, salvo la QNN en AUC.

Modelo	**Acc**	**F1**	**AUC**	**Gap**	t_{train} (s)	t_{pred} (s)
SVM (RBF)	0.871	0.870	0.936	0.021	0.003	0.007
Random Forest	0.854	0.854	0.943	0.146	0.078	0.006
XGBoost	0.850	0.849	0.953	0.067	0.074	0.001
QSVM	0.758	0.758	0.822	0.110	1.458	1.538
QNN	0.817	0.815	0.936	−0,006	5.614	0.006
VQC	0.696	0.688	—	0.014	12.432	1.872

Cuadro 5.2: Bars and Stripes, conjunto de test.

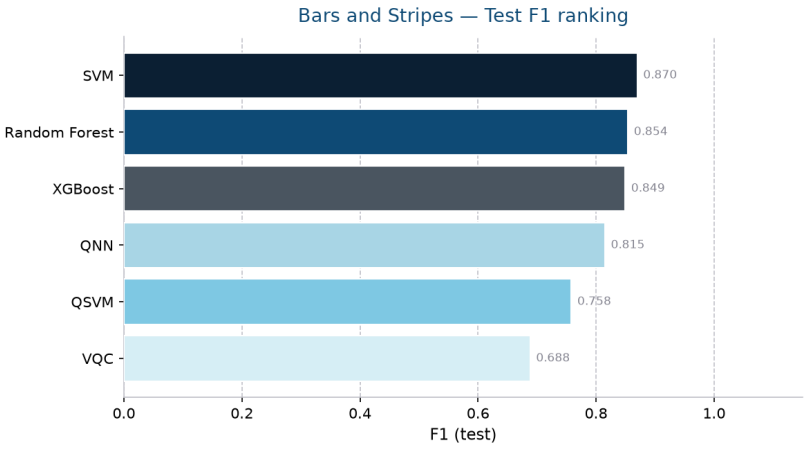


Figura 5.10: Ranking de F1 en Bars and Stripes. El SVM lidera; el VQC cierra la lista. *Fuente: elaboración propia.*

SVM

F1 0,870 y gap 0,021: el RBF sigue siendo el mejor compromiso. El AUC 0,936 muestra que, aunque el plano está mezclado, el kernel ordena razonablemente las dos clases. La

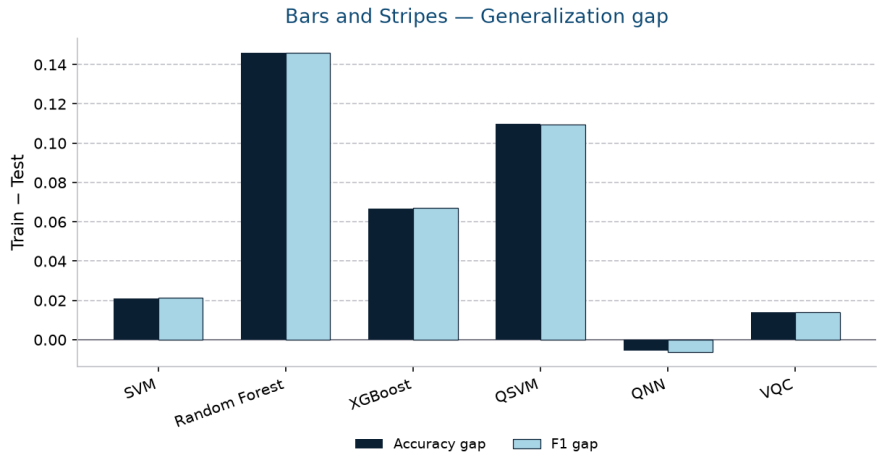


Figura 5.11: Gap de generalización en Bars and Stripes. El Random Forest es el que más se separa entre train y test.

Fuente: elaboración propia.

Figura 5.12 coloca la frontera sobre esa nube. Con d reducido a 4, el RBF opera en el mismo presupuesto de coordenadas que los circuitos y aun así generaliza mejor que ellos. El tiempo total permanece por debajo de la centésima de segundo.

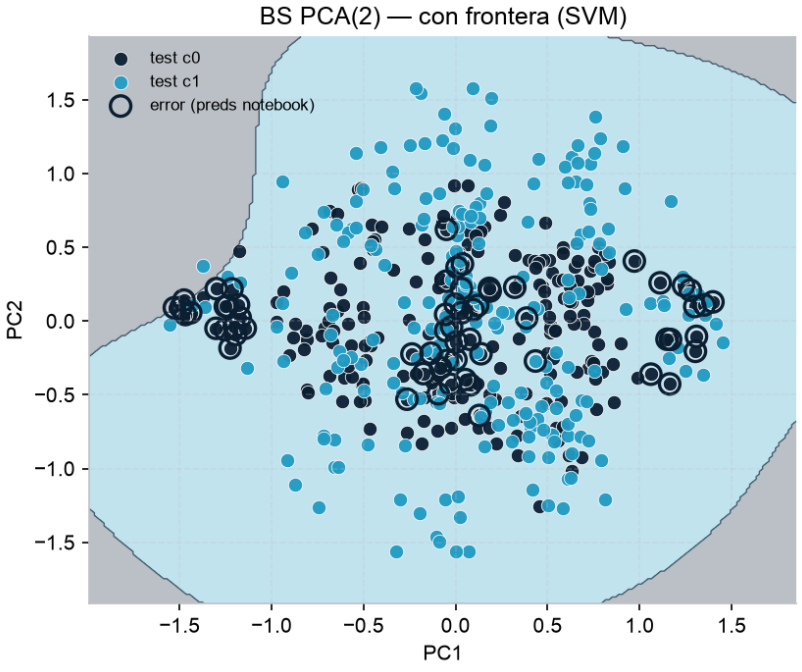


Figura 5.12: Frontera del SVM en Bars and Stripes. El plano mezcla las clases; el margen RBF es el que menos errores acumula.

Fuente: elaboración propia.

Random Forest

F1 0,854, AUC 0,943, gap 0,146. El bosque acierta en train y pierde en test: las reglas que aíslan barras y franjas en las hojas no sobreviven al 40 % de test. La Figura 5.13 es una partición fina de una nube sin margen. En un dato de patrones binarios comprimidos por PCA, el bagging memoriza combinaciones de componentes que no son estables. Es el caso claro en que accuracy de train no es la métrica de la comparación.

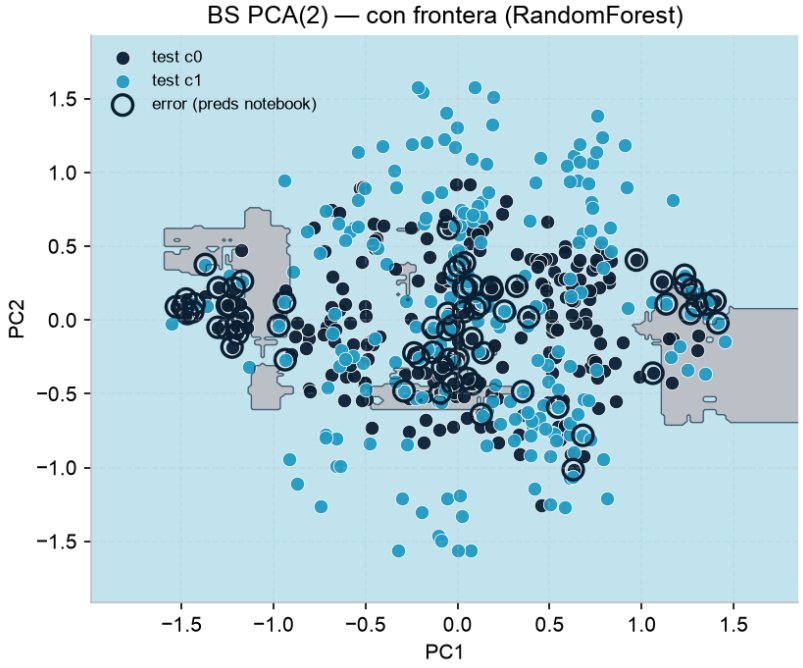


Figura 5.13: Frontera del Random Forest en Bars and Stripes. La partición ajusta el train; el gap de 0,146 mide lo que no se sostiene en test.
Fuente: elaboración propia.

XGBoost

F1 0,849, el AUC más alto del dataset (0,953) y un gap de 0,067, menos de la mitad que el bosque. El boosting regulariza la secuencia de árboles y ordena mejor las probabilidades, aunque la etiqueta dura quede un punto por debajo del SVM. La Figura 5.14 muestra esa frontera. La técnica aporta calibración (el AUC) más que accuracy: para un ranking de patrones, XGBoost es el clásico a mirar; para la etiqueta, el RBF sigue delante.

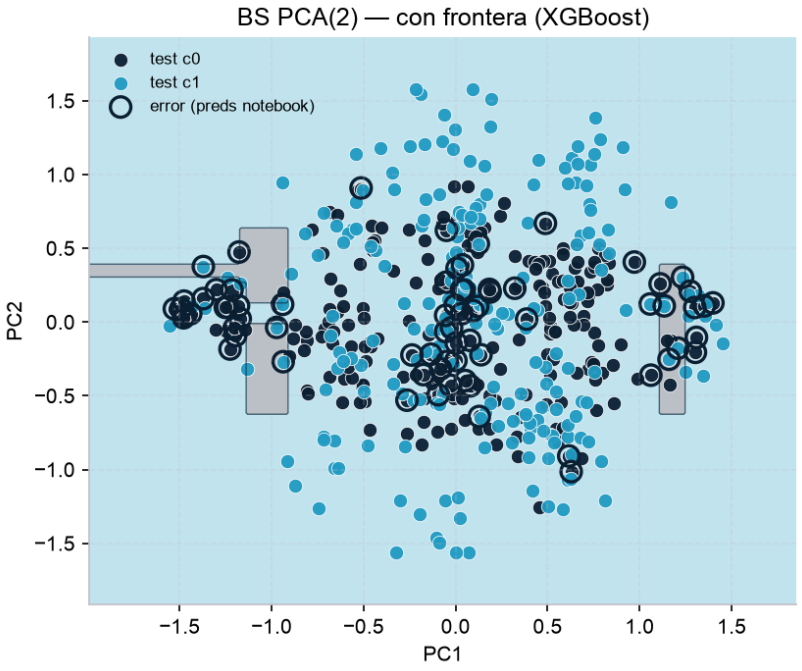


Figura 5.14: Frontera de XGBoost en Bars and Stripes. El F1 queda justo bajo el SVM; el AUC es el más alto de la tabla.

Fuente: elaboración propia.

QSVM

F1 0,758, AUC 0,822, gap 0,110. El mapa ZZ no encuentra en las cuatro componentes una correlación que separe barras de franjas mejor que un RBF. El coste cambia de escala: 1,46 s de ajuste y 1,54 s de inferencia, porque la matriz de fidelidad crece con el producto de tamaños de train y test y cada entrada es un solape de estados. La Figura 5.15 es la frontera inducida por ese kernel. En un dataset convencional, sin generador cuántico de las etiquetas, el feature map ZZ es una hipótesis inductiva cara y peor alineada (Huang et al., 2021; Schuld & Killoran, 2019).

QNN

Es el mejor modelo cuántico del dataset: F1 0,815, AUC 0,936 (el mismo que el SVM) y gap ligeramente negativo. La etiqueta dura queda 0,055 por debajo del SVM; el orden de los scores no. Angle embedding más entrelazamiento entrenable y una cabeza lineal leen una geometría que el kernel ZZ no linealiza igual. El entrenamiento dura 5,61 s; la inferencia, 0,006 s. La Figura 5.16 muestra la frontera. La discusión de técnica es la de un

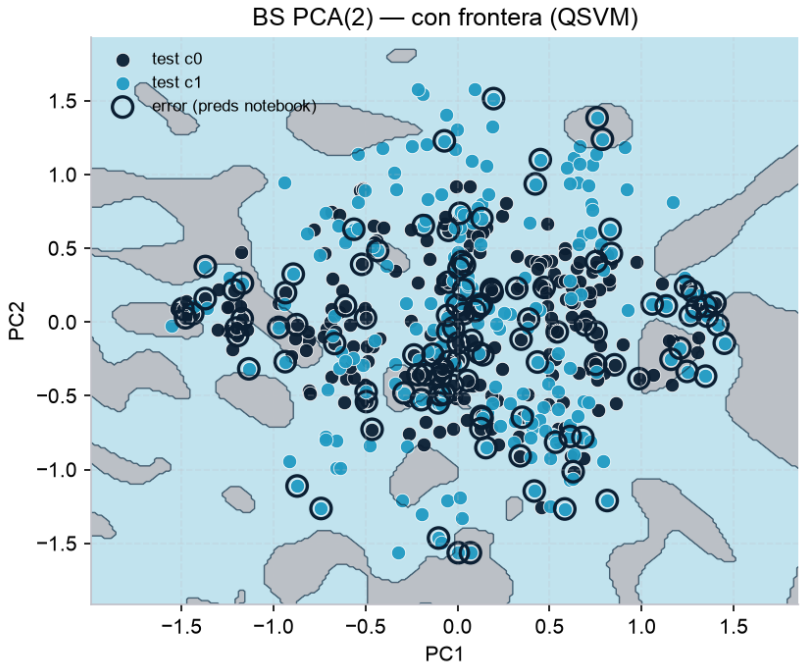


Figura 5.15: Frontera del QSVM en Bars and Stripes. El kernel de fidelidad no ordena la nube mejor que el RBF y multiplica el tiempo de inferencia.
Fuente: elaboración propia.

modelo híbrido: el circuito propone features $\langle Z_i \rangle$ y la cabeza clásica clasifica. En Bars and Stripes esa división acerca el AUC al mejor clásico y deja la accuracy todavía por debajo.

VQC

F1 0,688, el mínimo del corpus de clasificación, con 12,4 s de entrenamiento y 1,87 s de inferencia. La Figura 5.17 muestra un corte simple sobre la nube mezclada: el ansatz R_Y más `Rot`+CNOT, leído por $\langle Z \rangle$, no tiene grados de libertad para una frontera que siga el solape. El mismo circuito en Iris, donde una clase está separada, empataba con los árboles. Aquí la hipótesis del ansatz es demasiado rígida para el dato. Subir capas aumentaría el coste de simulación sin garantía de alinear el mapa con las barras (Cerezo et al., 2021).

5.1.3 Ising

Las features son amplitudes del estado fundamental de la cadena de cuatro espines de la Figura 4.5. La etiqueta es la fase, $y = \mathbf{1}[h > |J|]$, con transición en $h_c = |J| = 1$, y h no entra como variable. Es el único dataset en el que un modelo cuántico empata con el mejor clásico (Tabla 5.3). Aquí las fronteras sobre el plano principal no explican nada del

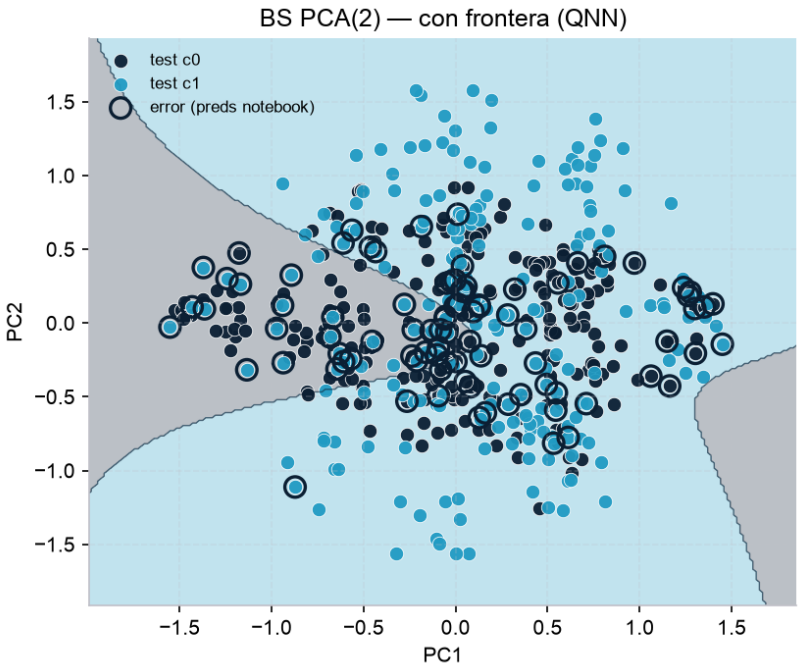


Figura 5.16: Frontera de la QNN en Bars and Stripes. El AUC empata con el SVM; la etiqueta dura no.

Fuente: elaboración propia.

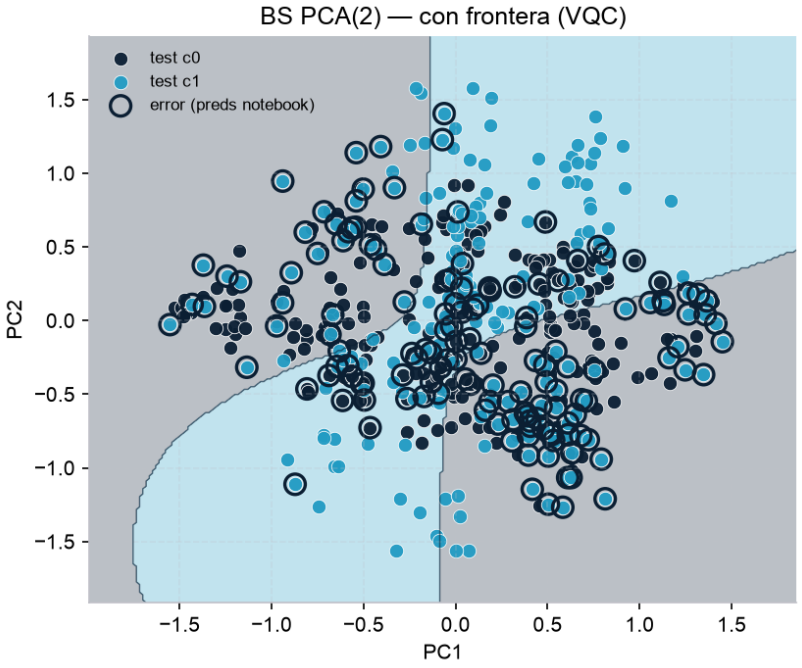


Figura 5.17: Frontera del VQC en Bars and Stripes. El corte deja la región de solape llena de errores de test.

Fuente: elaboración propia.

sistema, así que la lectura de cada implementación se apoya en la tabla y en el grafo: qué modelo reconoce la fase de ese Hamiltoniano y a qué coste.

Modelo	**Acc**	**F1**	**AUC**	**Gap**	t_{train} (s)	t_{pred} (s)
SVM (RBF)	0.988	0.985	1.000	0.013	0.001	0.0002
Random Forest	0.975	0.969	1.000	0.025	0.062	0.004
XGBoost	0.950	0.937	0.996	0.050	0.012	0.0005
QSVM	0.988	0.985	1.000	0.013	0.153	0.206
QNN	0.950	0.937	0.996	0.017	2.843	0.006
VQC	0.975	0.969	—	0.000	18.231	0.332

Cuadro 5.3: Ising, conjunto de test. Recall macro del SVM y del QSVM: 0,979.

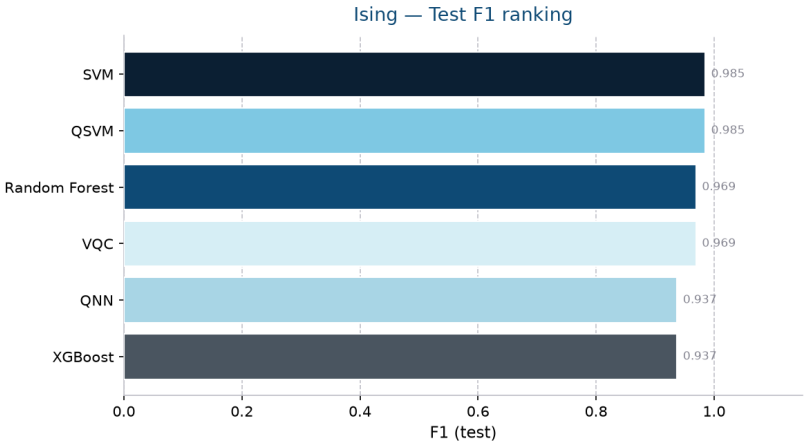


Figura 5.18: Ranking de F1 en Ising. SVM y QSVM comparten el primer puesto.
Fuente: elaboración propia.

SVM

F1 0,985, AUC 1,000, gap 0,013, en alrededor de un milisegundo. El recall macro (0,979) queda un punto bajo el accuracy: el error residual está en una de las dos fases. El RBF no necesita saber que el vector viene de un estado cuántico: las amplitudes del estado fundamental cambian lo bastante con h como para que un margen clásico separe orden y desorden. Por eso el empate posterior del QSVM no es, por sí solo, una ventaja operativa: el mismo F1 llega antes por el kernel clásico.

Random Forest

F1 0,969 y AUC 1,000, con gap 0,025. El bosque ordena las fases tan bien como el SVM en AUC y pierde algo de F1 en la etiqueta dura. Los umbrales sobre componentes de las amplitudes bastan para distinguir la fase; el bagging no añade mucho sobre un solo margen. El tiempo (0,062 s) sigue siendo inferior al de cualquier circuito.

XGBoost

F1 0,937, recall macro 0,917, AUC 0,996, gap 0,050. Es el clásico que más se deja una de las fases. El boosting, con este tamaño de muestra, sobreajusta el corte cerca de la transición. El dato premia un margen global; una secuencia de árboles cortos no es esa hipótesis. Aun así el ajuste dura 0,012 s.

QSVM

Empata con el SVM en Acc 0,988, F1 0,985, AUC 1,000 y gap 0,013. Aquí el mapa ZZ sí está cerca del generador: las features son amplitudes del estado de la cadena de la Figura 4.5 y la etiqueta es la fase de ese mismo Hamiltoniano, cuyas interacciones son precisamente términos Z_iZ_{i+1} (Huang et al., 2021; Havlíček et al., 2019). El empate de calidad no arrastra al tiempo. El ajuste dura 0,153 s y la predicción 0,206 s, frente a 0,001 s del SVM. La discusión queda en la alineación dato–mapa: cuando el kernel cuántico está en el espacio correcto, recupera al clásico; la simulación del solape sigue costando más que evaluar un RBF.

QNN

F1 0,937 y AUC 0,996, las mismas cifras que XGBoost, con gap menor (0,017). La cabeza lineal sobre angle embedding no aprovecha la estructura de fase mejor que un boosting corto, y el entrenamiento sube a 2,84 s. En un dato donde el kernel ZZ ya empata con el RBF, cambiar a un embedding angular separable y entrenar capas es una hipótesis más débil: se pierde la interacción Z_iZ_j que refleja los acoplamientos de la cadena.

VQC

F1 0,969, el mismo que el Random Forest, gap 0,000 y 18,2 s de entrenamiento. La preparación R_Y más el ansatz variacional distingue las fases sin el sobreajuste de XGBoost,

pero no alcanza el empate SVM–QSVM. Cada predicción reejecuta el circuito (0,332 s). Frente al QSVM, el VQC no usa un mapa con los acoplamientos Z_iZ_j de la cadena; frente al SVM, repite una calidad intermedia a un coste de simulación que en este tamaño no se justifica (Bowles et al., 2024).

5.1.4 Lectura de la familia

Tres técnicas, tres resultados. El kernel RBF es el mejor o está empatado en los tres datasets y es el más barato. Los árboles se acercan cuando la geometría es simple (Iris, Ising) y generalizan peor cuando la nube está mezclada (Bars and Stripes, gap del bosque 0,146). El mapa ZZ solo empata con el RBF en Ising, donde las features ya son un estado de un Hamiltoniano y la etiqueta es su fase; en Iris fragmenta la frontera y en Bars and Stripes paga segundos de fidelidad por un F1 inferior (Huang et al., 2021). La QNN híbrida es el cuántico más estable fuera de Ising: su AUC en Bars and Stripes iguala al SVM, porque la cabeza clásica clasifica $\langle Z_i \rangle$. El VQC reproduce la calidad de los árboles en Iris e Ising y cae en Bars and Stripes, siempre con el mayor tiempo de entrenamiento. En simulación de $n = 4$, la comparación cuantitativa no muestra una región en la que el circuito mejore al RBF en F1 y en tiempo a la vez (Bowles et al., 2024).

5.2 Optimización

Las instancias tienen óptimo conocido y caben en enumeración: TSP con coste $C^\star = 16$, MDS con $C^\star = 2$, MaxCut con $C^\star = 8$. La Figura 5.19 sitúa el success rate medio frente al tiempo. El detalle, y las dos excepciones (greedy en MDS, QAOA en MDS, SA en TSP), está en las tablas de cada problema. En este tamaño, alcanzar $C^\star$ es lo esperado de un método exacto; lo que distingue a los solvers es el camino y el wall-clock (Lubinski et al., 2024; Lucas, 2014).

5.2.1 TSP

Grafo completo de 5 nodos. El óptimo por enumeración es 16. La Tabla 5.4 y la Figura 5.20 resumen los seis solvers.

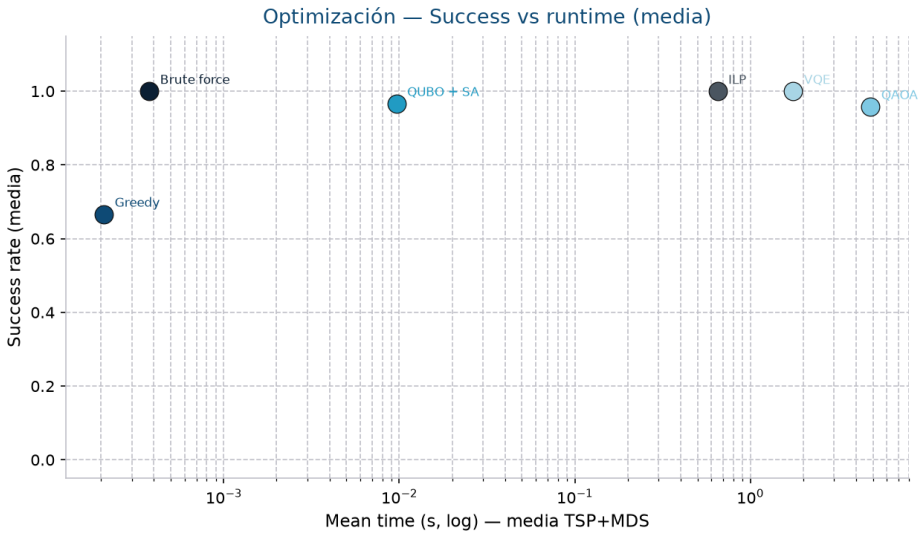


Figura 5.19: Success rate medio frente al tiempo medio. La fuerza bruta es exacta y barata en estas instancias; QAOA y VQE se sitúan en segundos de simulación.

Fuente: elaboración propia.

Solver	**Éxito**	C medio	C mejor	**Gap**	**Gap rel.**	t (s)
Fuerza bruta	1.00	16.0	16	0.00	0.000	0.0009
Greedy	1.00	16.0	16	0.00	0.000	0.0005
ILP	1.00	16.0	16	0.00	0.000	1.277
QUBO + SA	0.90	16.3	16	0.30	0.019	0.017
QAOA	1.00	16.0	16	0.00	0.000	1.059
VQE	1.00	16.0	16	0.00	0.000	1.909

Cuadro 5.4: TSP. $C^\star = 16$. El SA promedia semillas; el resto alcanza el óptimo en las ejecuciones reportadas.

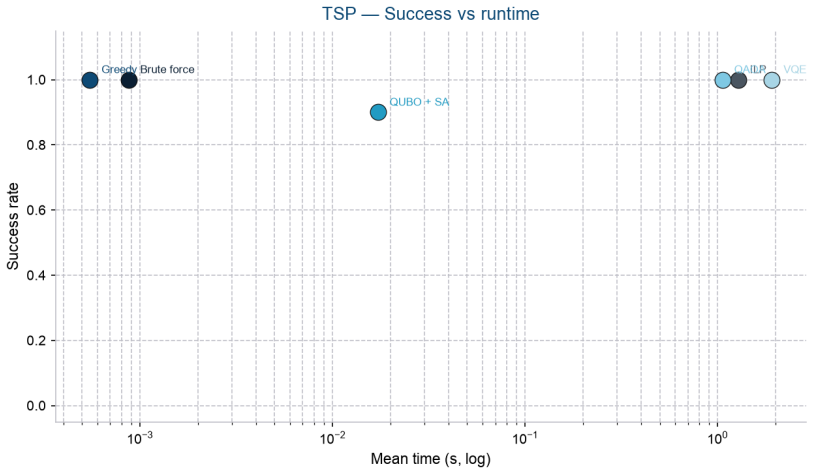


Figura 5.20: TSP: success rate frente al tiempo. El SA es el único que no satura el éxito; ILP, QAOA y VQE comparten la escala del segundo.

Fuente: elaboración propia.

Fuerza bruta

Enumera las rutas y devuelve $C^\star$ en menos de un milisegundo (6 evaluaciones de la función de coste). La Figura 5.21 es una ruta óptima. Con 5 nodos el espacio de tours es el método de referencia, no un baseline débil: cualquier otro solver se juzga contra una enumeración que ya es barata.

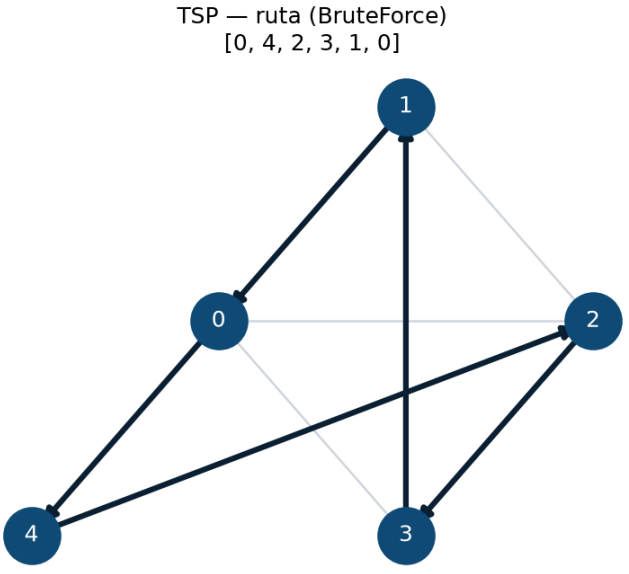


Figura 5.21: Ruta óptima del TSP por enumeración. Coste 16.

Fuente: elaboración propia.

Greedy

La heurística de vecino más cercano también cierra en 16 (Figura 5.22), en 0,5 ms. En este grafo el óptimo local coincide con el global. No es una propiedad del algoritmo: en MDS, con la misma heurística, el conjunto se pasa de tamaño. Aquí el grafo es pequeño y las distancias no castigan el primer paso miope.

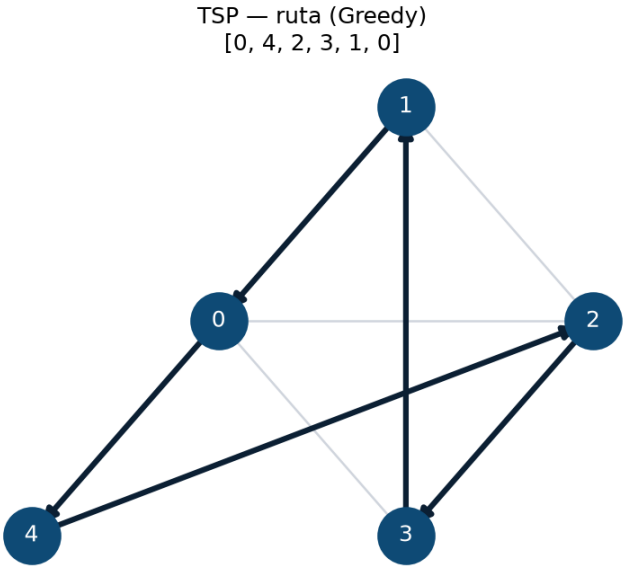


Figura 5.22: Ruta del greedy en el TSP. El coste coincide con el óptimo.
Fuente: elaboración propia.

ILP

La formulación lineal con CBC devuelve el óptimo y tarda 1,28 s. Ese tiempo es de arranque del solver, no de una búsqueda larga: la fuerza bruta, sobre el mismo espacio, termina en menos de un milisegundo. La Figura 5.23 certifica la ruta. El ILP importa como método exacto general; en esta instancia no es el exacto más rápido.

QUBO + simulated annealing

El annealing sobre el QUBO acierta el óptimo en el 90 % de las semillas. La media de coste sube a 16,3 (gap relativo 0,019) con unas 5000 evaluaciones y 0,017 s. La Figura 5.24 muestra una semilla óptima, la ruta $[0, 4, 2, 3, 1, 0]$. La técnica es clásica y opera en la misma energía que QAOA y VQE: cuando la penalización del QUBO está bien puesta, el recocido encuentra el tour; una semilla de cada diez cae en un coste vecino. Es el único

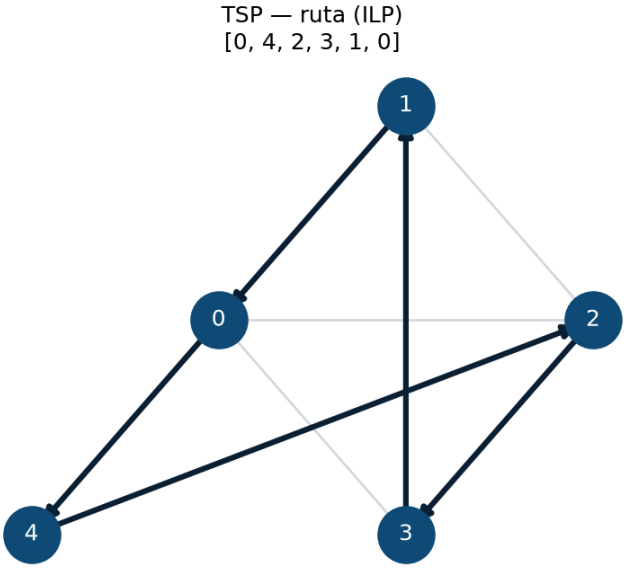


Figura 5.23: Ruta del ILP en el TSP. Óptimo de coste 16, con el mayor tiempo clásico de la tabla.

Fuente: elaboración propia.

solver del TSP que no tiene éxito 1, y sigue siendo dos órdenes más rápido que el circuito simulado.

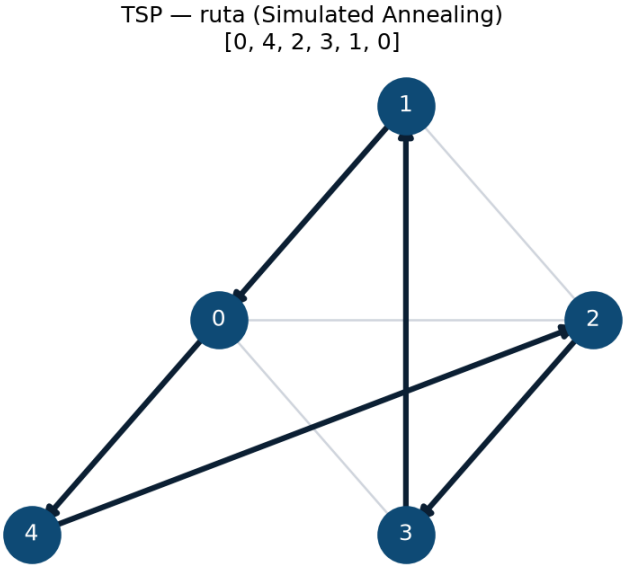


Figura 5.24: Una ruta óptima del simulated annealing, $[0, 4, 2, 3, 1, 0]$. El success rate sobre semillas es 0,90.

Fuente: elaboración propia.

QAOA

Capas $U_C U_B$ sobre el mismo QUBO, en simulación de estado, alcanzan coste 16 en las ejecuciones reportadas (éxito 1, unas 356 evaluaciones, 1,06 s). La Figura 5.25 es el tour devuelto. QAOA reproduce el óptimo que la enumeración ya tiene, en un tiempo del orden del ILP y muy por encima del annealing. Con 5 nodos el circuito no explora un espacio que el clásico no enumere; mide si el ansatz variacional concentra probabilidad en la ruta óptima (Farhi et al., 2014). Aquí sí. El coste es el de simular esas capas.

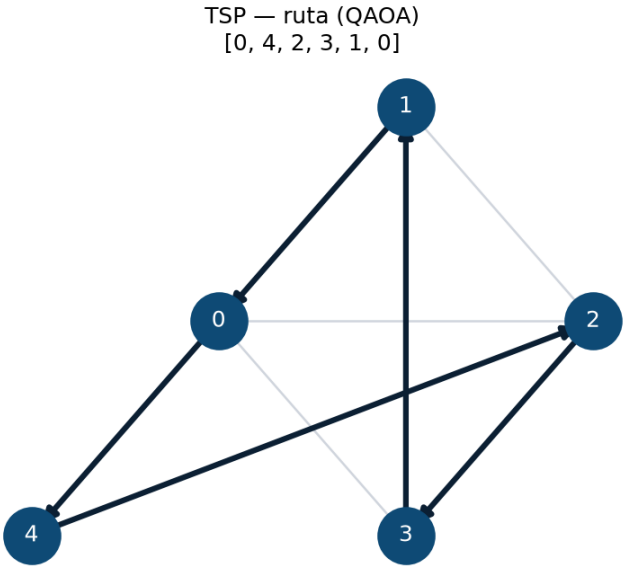


Figura 5.25: Ruta devuelta por QAOA en el TSP. Coste óptimo, en torno a un segundo de simulación.

Fuente: elaboración propia.

VQE

El ansatz variacional minimiza la energía del Ising equivalente y también devuelve coste 16 (Figura 5.26), con unas 720 evaluaciones y 1,91 s, el mayor tiempo del problema. La diferencia con QAOA no está en la calidad, que satura, sino en el número de evaluaciones del Hamiltoniano y en que el ansatz no está estructurado como la alternancia coste–mezclador. En una instancia enumerable, VQE es una comprobación de que el óptimo es un mínimo del QUBO, no un algoritmo más corto que la fuerza bruta (Peruzzo et al., 2014).

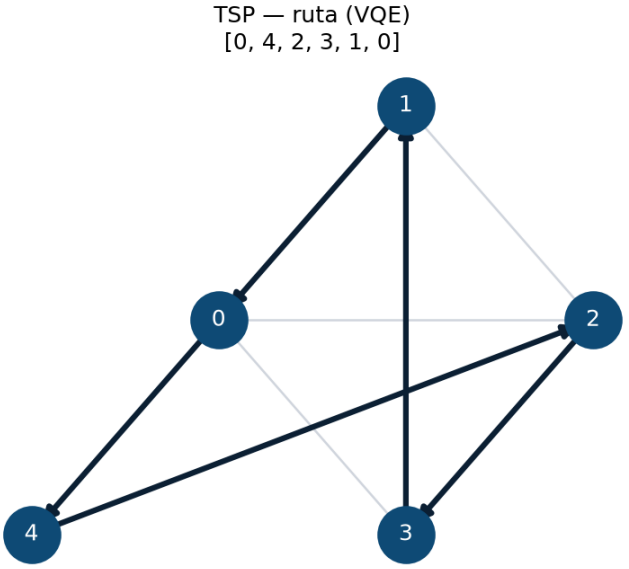


Figura 5.26: Ruta devuelta por VQE en el TSP. Mismo coste óptimo, más evaluaciones de energía que QAOA.

Fuente: elaboración propia.

5.2.2 Minimum dominating set

Grafo de 6 vértices y 6 aristas. El óptimo es un conjunto de tamaño 2. Es la instancia que separa heurística, annealing y circuito (Tabla 5.5).

Solver	**Éxito**	C medio	C mejor	**Gap**	**Gap rel.**	t (s)
Fuerza bruta	1.00	2.00	2	0.00	0.000	0.0002
Greedy	0.00	3.00	3	1.00	0.500	$<$ 0,001
ILP	1.00	2.00	2	0.00	0.000	0.666
QUBO + SA	1.00	2.00	2	0.00	0.000	0.008
QAOA	0.875	2.25	2	0.25	0.125	12.232
VQE	1.00	2.00	2	0.00	0.000	1.561

Cuadro 5.5: MDS. $C^\star = 2$. QAOA promedia semillas; el mejor coste de esas semillas sigue siendo 2.

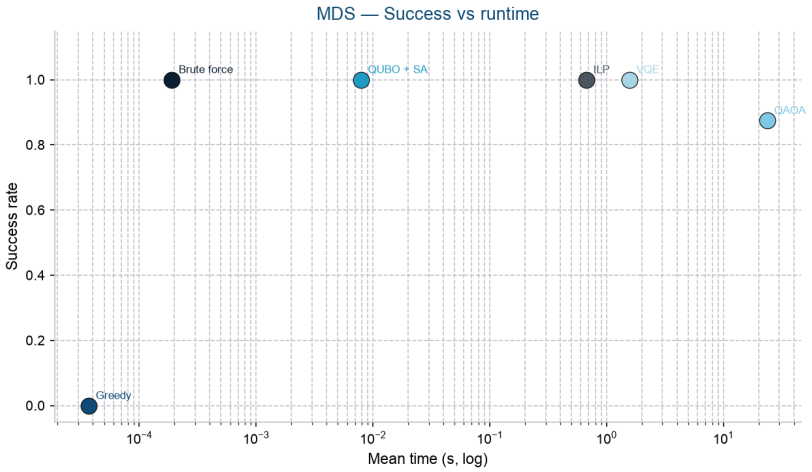


Figura 5.27: MDS: success rate frente al tiempo. Greedy no acierta; QAOA acierta en una fracción de las semillas y es el más lento.

Fuente: elaboración propia.

Fuerza bruta

Enumera subconjuntos y fija $C^{\star} = 2$ en 0,2 ms. La Figura 5.28 es un conjunto dominante mínimo. Con 6 vértices, $2^6 = 64$ máscaras: el óptimo de referencia sale antes de que un solver iterativo arranque.

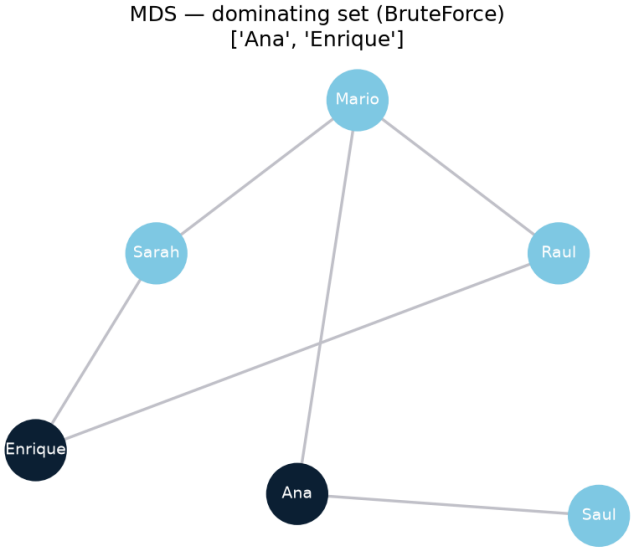


Figura 5.28: Conjunto dominante mínimo por enumeración. Tamaño 2.

Fuente: elaboración propia.

Greedy

Elige Mario, Sarah y Ana (Figura 5.29): tamaño 3, gap 1, gap relativo 0,5, éxito 0. La heurística cubre por grado local y no deshace la primera elección. Enrique y Ana solos dominan el grafo; el greedy no llega a ese par porque ya ha gastado un vértice. Es el único fallo sistemático de la familia clásica y no se arregla con más tiempo: la regla es determinista. En grafos de cobertura, un voraz sin backtracking no es un baseline de calidad, es una cota rápida que aquí queda corta.

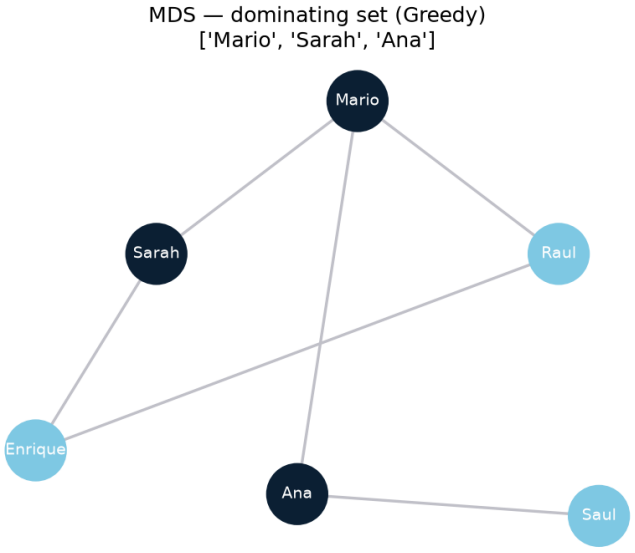


Figura 5.29: Conjunto del greedy: Mario, Sarah y Ana. Tamaño 3, uno por encima del óptimo.

Fuente: elaboración propia.

ILP

La cobertura lineal con CBC devuelve un conjunto de tamaño 2 en 0,67 s (Figura 5.30). El modelo es exacto: cada vértice queda dominado y la objetivo minimiza la suma de binarias. El tiempo, como en el TSP, incluye el arranque de CBC. Frente al greedy, el ILP paga décimas de segundo y cierra el gap. Frente a la enumeración, es más lento y igual de correcto.

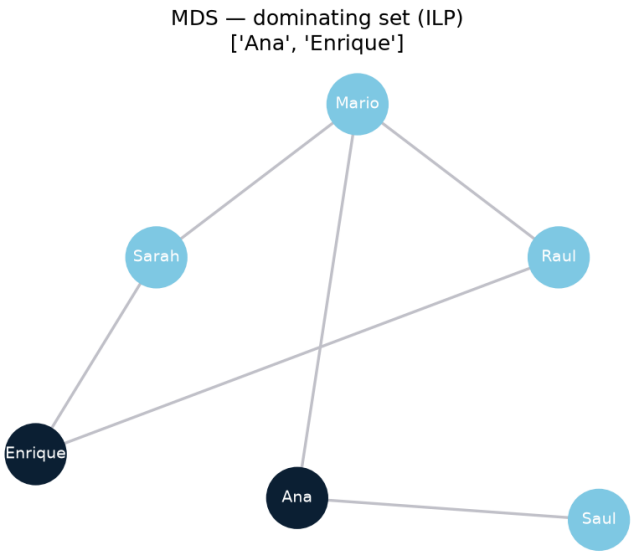


Figura 5.30: Conjunto dominante del ILP. Tamaño óptimo 2.

Fuente: elaboración propia.

QUBO + simulated annealing

El annealing sobre el QUBO de cobertura acierta tamaño 2 en todas las semillas reportadas, con unas 4000 evaluaciones y 0,008 s (Figura 5.31). La misma energía que luego ven QAOA y VQE ya es minimizable por recocido clásico. En esta instancia la formulación QUBO no deforma el óptimo: si la penalización de no-dominancia está bien escalada, el SA cae en $C^\star$ (Lucas, 2014). El contraste con el TSP, donde una semilla fallaba, es de paisaje, no de método.

QAOA

Es el resultado cuántico-inspirado que no satura. Éxito 0,875, coste medio 2,25, gap 0,25, unas 4000 evaluaciones y 12,2 s, el máximo de todo el corpus de optimización. La Figura 5.32 es una semilla óptima: Ana y Enrique, tamaño 2. Otras semillas salen con un vértice de más, y por eso la media no es 2. El ansatz $U_C U_B$ ve el mismo QUBO que el SA, que sí acierta siempre y en milisegundos. La discusión es de optimización variacional, no de formulación: con pocas capas y en simulación, la distribución de medida no concentra toda la masa en el bitstring óptimo (Farhi et al., 2014; Lubinski et al., 2024). Subir capas alargaría aún más la simulación.

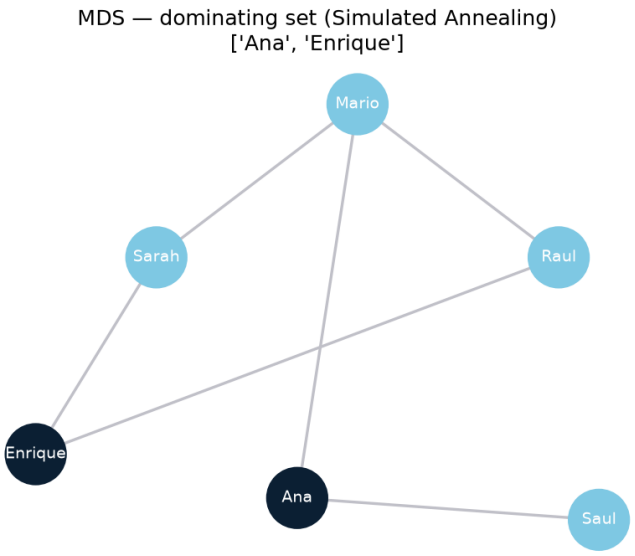


Figura 5.31: Conjunto devuelto por simulated annealing. Tamaño 2 en las semillas reportadas.

Fuente: elaboración propia.

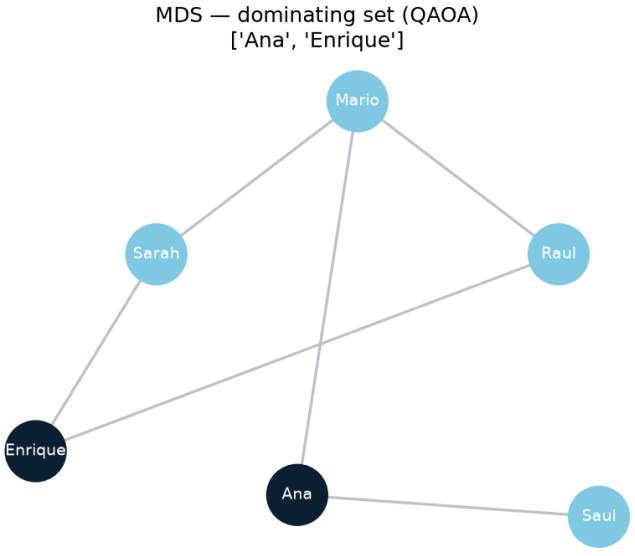


Figura 5.32: Una ejecución óptima de QAOA: Ana y Enrique. El success rate sobre semillas es 0,875.

Fuente: elaboración propia.

VQE

Alcanza tamaño 2 en las semillas reportadas (Figura 5.33), con unas 600 evaluaciones y 1,56 s. El mismo problema en el que QAOA oscila, el VQE lo cierra. El ansatz y el número de evaluaciones no son los de QAOA: aquí bastan menos llamadas al Hamiltoniano para clavar el mínimo que el SA ya encontraba. Sigue siendo dos órdenes más lento que el annealing y tres que la enumeración. La técnica confirma el mínimo del Ising; no acorta la búsqueda en un grafo de 6 vértices (Peruzzo et al., 2014).

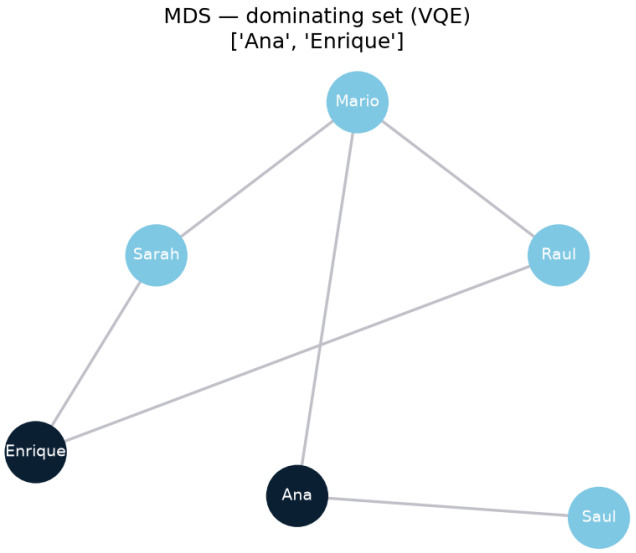


Figura 5.33: Conjunto devuelto por VQE. Tamaño óptimo, por debajo del tiempo de QAOA y por encima del SA.
Fuente: elaboración propia.

5.2.3 MaxCut

Instancia HamLib de 6 vértices y 12 aristas, corte óptimo 8. Los seis solvers lo alcanzan (Tabla 5.6). La figura de cada implementación es la partición que realiza ese corte.

Fuerza bruta

Evalúa las 2^6 biparticiones y devuelve corte 8 en decenas de microsegundos (Figura 5.35). En MaxCut el espacio es el de asignar un bit por vértice, exactamente el espacio de un circuito de 6 qubits, y la enumeración lo recorre entero. El óptimo de referencia queda fijado antes de hablar de ansatz.

Solver	**Éxito**	C medio	C mejor	**Gap**	**Gap rel.**	t (s)
Fuerza bruta	1.00	8	8	0	0	$7 \cdot 10^{-5}$
Greedy	1.00	8	8	0	0	$4 \cdot 10^{-5}$
ILP	1.00	8	8	0	0	0.011
QUBO + SA	1.00	8	8	0	0	0.004
QAOA	1.00	8	8	0	0	1.247
VQE	1.00	8	8	0	0	1.799

Cuadro 5.6: MaxCut HamLib. $C^\star = 8$. Todos los solvers reportados alcanzan el corte óptimo.

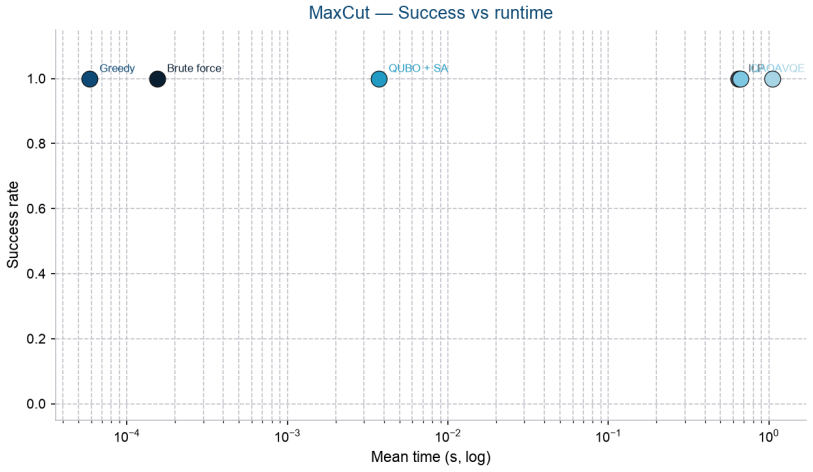


Figura 5.34: MaxCut: éxito 1 para todos los solvers. El eje de tiempo separa enumeración, greedy y SA del ILP y, más allá, de QAOA y VQE.

Fuente: elaboración propia.

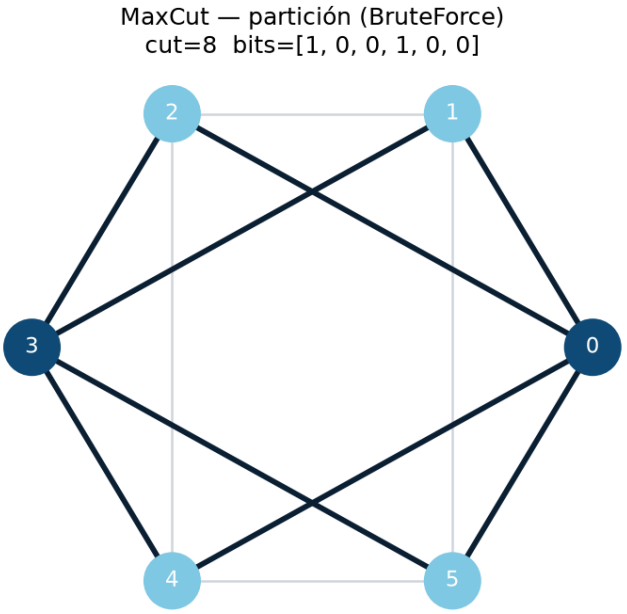


Figura 5.35: Corte óptimo de MaxCut por enumeración. Valor 8.

Fuente: elaboración propia.

Greedy

También corta 8 (Figura 5.36), aún más rápido. A diferencia del MDS, en esta instancia el crecimiento local de la partición no se queda corto. El grafo es denso (12 aristas sobre 6 vértices) y varios cortes alcanzan el óptimo, así que una heurística miope tiene donde acertar. No se generaliza al MDS, donde el mismo tipo de regla falló.

ILP

CBC devuelve corte 8 en 0,011 s (Figura 5.37). El modelo lineal de MaxCut es compacto en 6 vértices y el arranque del solver pesa menos que en el TSP. Sigue siendo exacto y más lento que enumerar 64 asignaciones. Su papel en el protocolo es el método exacto de formulación nativa, al lado del QUBO que usan SA, QAOA y VQE.

QUBO + simulated annealing

Corte 8 en todas las semillas, unas 4000 evaluaciones, 0,004 s (Figura 5.38). El QUBO de MaxCut no lleva restricciones de igualdad duras: la objetivo es el corte, y el paisaje que ve el SA es el mismo que el Hamiltoniano de coste de QAOA. Aquí el recocido clásico encuentra el mínimo que el circuito también encuentra, tres órdenes de magnitud antes.

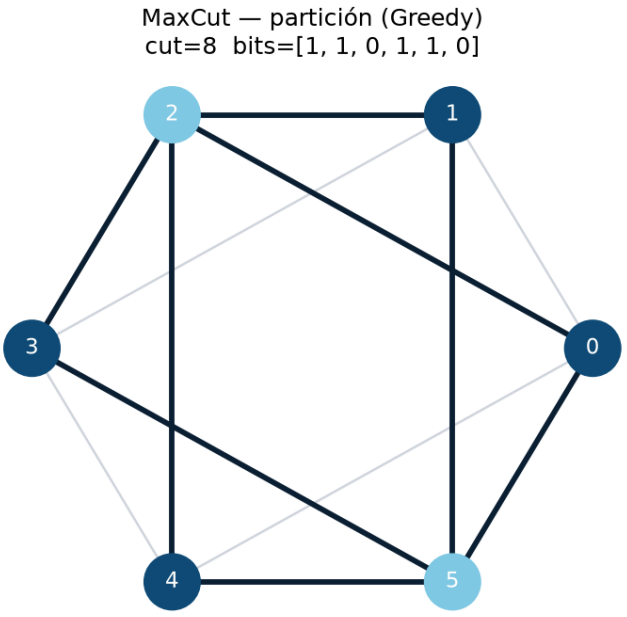


Figura 5.36: Corte del greedy en MaxCut. Valor óptimo 8.

Fuente: elaboración propia.

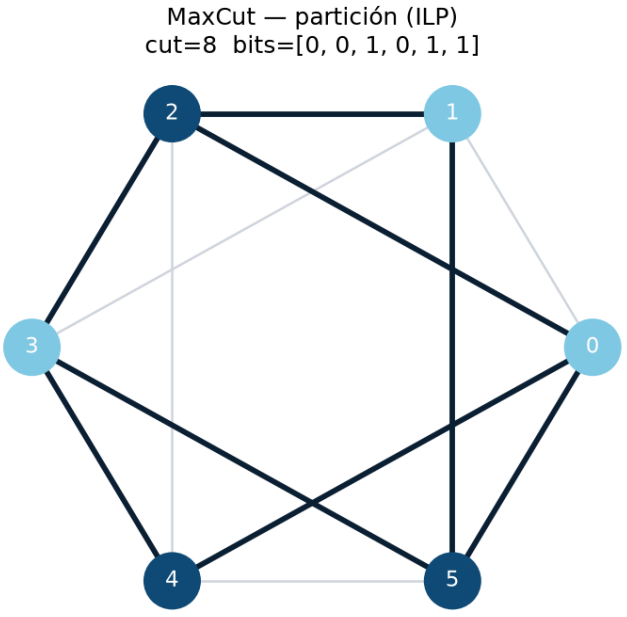


Figura 5.37: Corte del ILP en MaxCut. Valor 8.

Fuente: elaboración propia.

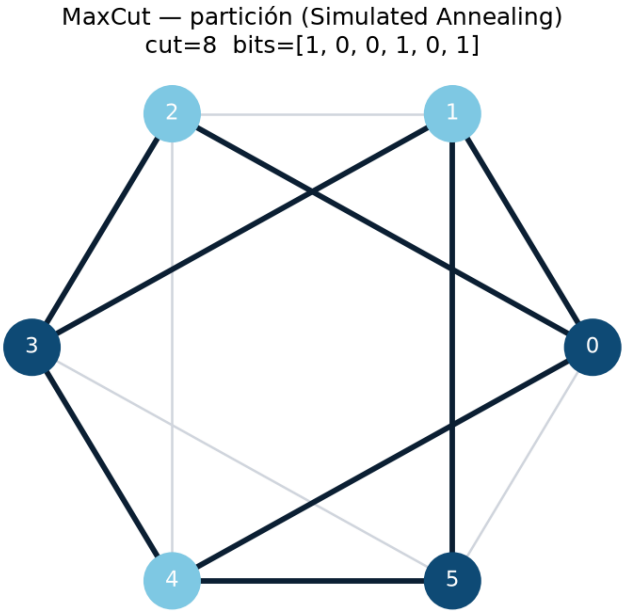


Figura 5.38: Corte del simulated annealing en MaxCut. Valor óptimo en las semillas reportadas.
Fuente: elaboración propia.

QAOA

Corte 8, éxito 1, unas 40 evaluaciones, 1,25 s (Figura 5.39). Pocas evaluaciones bastan porque el problema es el ejemplo estándar del ansatz: U_C es el Hamiltoniano del corte y U_B mezcla las biparticiones (Farhi et al., 2014; PennyLane, n.d.j). La calidad empata con la enumeración. El tiempo es el de simular el estado de 6 qubits, no el de probar 64 cortes. En la comparación cuantitativa, QAOA demuestra que el circuito codifica el problema; no que lo resuelva antes que recorrerlo.

VQE

Corte 8, unas 240 evaluaciones, 1,80 s (Figura 5.40). Más llamadas al Hamiltoniano que QAOA para el mismo corte: el ansatz genérico no trae la alternancia coste–mezclador que ya es el algoritmo de MaxCut. La calidad no distingue a los dos. El número de evaluaciones sí, y ambos quedan lejos de los $7 \cdot 10^{-5}$ s de la enumeración (Peruzzo et al., 2014; Lubinski et al., 2024).

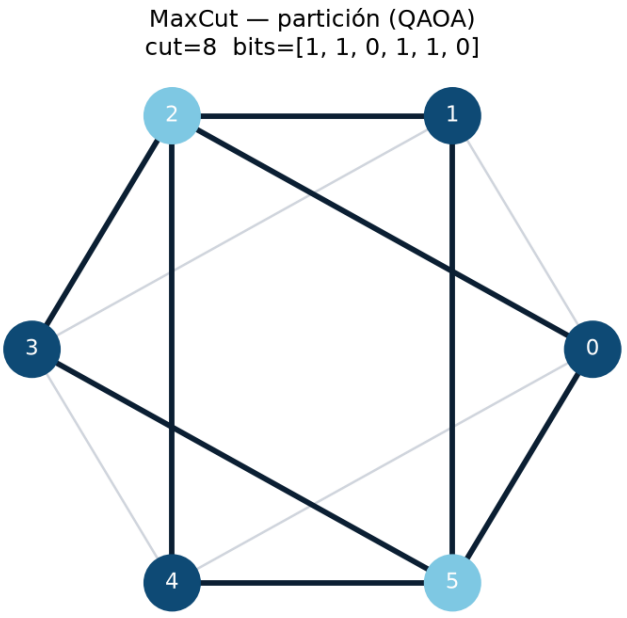


Figura 5.39: Corte devuelto por QAOA. Valor óptimo 8, en torno a 1,2 s de simulación.

Fuente: elaboración propia.

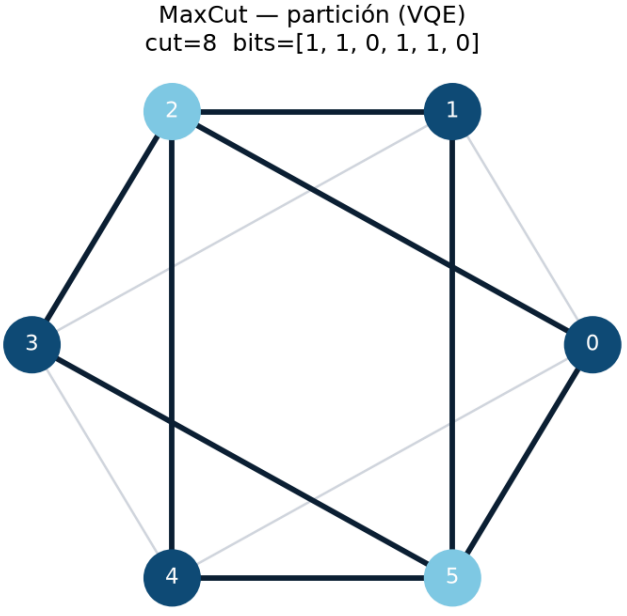


Figura 5.40: Corte devuelto por VQE. Mismo óptimo que QAOA, con más evaluaciones de energía.

Fuente: elaboración propia.

5.2.4 Lectura de la familia

En las tres instancias el óptimo es alcanzable por enumeración en menos de un milisegundo. El ILP también es exacto; su wall-clock mide el arranque de CBC, que en el TSP llega al segundo y en MaxCut baja a centésimas. El greedy solo falla en MDS, donde la elección local produce un conjunto de tamaño 3 y el óptimo es 2. El SA sobre QUBO empata con el óptimo en MDS y MaxCut y se deja una semilla de cada diez en el TSP, siempre en centésimas de segundo. QAOA y VQE, en simulación, recuperan $C^\star$ salvo QAOA en MDS (éxito 0,875, 12,2 s). Cuando aciertan, el circuito está bien planteado sobre el QUBO; el tiempo es el de evaluar el ansatz, no una búsqueda más corta que 2^n en $n \leq 6$ (Farhi et al., 2014; Lubinski et al., 2024). La lectura conjunta de calidad y latencia, en este régimen, es que el óptimo ya está en los métodos exactos clásicos y en el annealing, y que el circuito lo reproduce con un coste de simulación mayor.

5.3 Discusión: la representación como eje

Las dos familias se pueden leer con la misma pregunta: en qué espacio se resuelve el problema y si ese espacio está alineado con la estructura del dato o de la instancia (Lloyd et al., 2020; Huang et al., 2021; Parzygnat et al., 2025).

En clasificación. El feature map decide más que el modelo que lo lee. El mapa ZZ solo empata con el RBF en Ising, el único dataset de sistema. Allí las features son amplitudes del estado fundamental de la cadena y el Hamiltoniano tiene los mismos términos Z_iZ_{i+1} que el mapa entrelaza. En Iris y Bars and Stripes, datasets convencionales, el mismo mapa fragmenta la frontera o pierde F1 frente al RBF (Havlíček et al., 2019; Huang et al., 2021). Los embeddings angulares separables (QNN, VQC) son más estables fuera de Ising, pero no aportan una geometría que el kernel clásico no tenga ya. En este corpus, la representación cuántica rinde cuando el generador de etiquetas es físico y comparte estructura con el mapa; no cuando el dato es clásico (Bowles et al., 2024).

En optimización. SA, QAOA y VQE ven el mismo QUBO. Cuando la penalización está bien puesta, el annealing clásico encuentra $C^\star$ sobre esa energía en centésimas de segundo, lo que certifica la formulación. Las diferencias aparecen después de la representación. En MaxCut, sin restricciones, el Hamiltoniano de coste es el corte y QAOA lo resuelve con

pocas evaluaciones (Farhi et al., 2014). En MDS, con penalizaciones de cobertura, QAOA no concentra toda la probabilidad en el óptimo (éxito 0,875), mientras que el SA sobre la misma formulación sí. La dificultad no está en el QUBO, sino en optimizar el ansatz sobre un paisaje con restricciones (Lucas, 2014; Lubinski et al., 2024).

Límite del régimen. Con $n = 4$ qubits en clasificación y $n \leq 6$ en optimización, el experimento mide cercanía al clásico, no ventaja asintótica (Preskill, 2018; Thanasilp et al., 2024). Los resultados dan paridad o déficit en accuracy y F1, y óptimos que los solvers clásicos ya alcanzan, con mayor wall-clock en QAOA y VQE simulados (Bowles et al., 2024; Lubinski et al., 2024). Tres preguntas siguen abiertas: si la alineación dato–mapa observada en Ising se mantiene al crecer la cadena; si un QUBO de mayor tamaño separa a QAOA del annealing; y qué parte del coste temporal corresponde a la simulación de estado y cuál al método en sí.

6. Conclusiones

El trabajo partió de una pregunta: si los algoritmos cuánticos de la era NISQ ofrecen una ventaja medible frente a los clásicos en problemas de ciencia de datos. Para responderla se compararon seis modelos de clasificación sobre Iris, Bars and Stripes e Ising y seis solvers de optimización sobre TSP, MDS y MaxCut, con un protocolo común y la representación como eje de lectura. Las conclusiones siguen de los resultados del Capítulo 5.

6.1 Conclusiones

1. **En clasificación no se observa ventaja cuántica.** El SVM con kernel RBF obtiene el mejor F1 o empata con el mejor en los tres datasets (0.967 en Iris, 0.870 en Bars and Stripes, 0.985 en Ising). Ningún modelo cuántico lo supera.

2. **El rendimiento del modelo cuántico lo decide el feature map, no el clasificador.** El QSVM con mapa ZZ solo empata con el SVM en Ising (F1 0.985), donde las features proceden de un Hamiltoniano con los mismos acoplamientos Z_iZ_{i+1} que el mapa entrelaza. En Iris (0.730) y Bars and Stripes (0.758) cae claramente, de acuerdo con la alineación dato–mapa descrita por Huang et al. (2021).

3. **Los embeddings angulares son más estables, pero no aportan geometría nueva.** El QNN híbrido es el modelo cuántico más regular fuera de Ising (F1 0.933 en Iris y 0.815 en Bars and Stripes, con brecha de generalización casi nula). El VQC queda el último en Bars and Stripes (0.688) y es el más caro de entrenar en los tres datasets.

4. **En optimización, QAOA y VQE reproducen el óptimo, pero no mejoran a los métodos clásicos.** Los métodos exactos alcanzan $C^\star$ en las tres instancias, y el annealing simulado lo hace sobre la misma formulación QUBO en centésimas de segundo (éxito 0.9 en TSP y 1 en MDS y MaxCut), lo que valida la formulación. QAOA y VQE llegan al óptimo en TSP y MaxCut, pero tardan entre 1 y 2 s de simulación. En MDS, QAOA se queda en un éxito de 0.875 por la dificultad de optimizar el ansatz sobre un paisaje con penalizaciones, no por el QUBO.

5. **El régimen estudiado mide proximidad al clásico, no ventaja.** Con 4 qubits en clasificación, como mucho 6 en optimización y simulación de estado exacta, los

resultados no permiten afirmar ni descartar una ventaja a escala (Preskill, 2018; Thanasilp et al., 2024). Sí permiten afirmar que, en este rango, cualquier ventaja exige una representación alineada con la estructura del problema.

6.2 Cumplimiento de objetivos

La Tabla 6.1 relaciona cada objetivo específico del Capítulo 3 con el lugar del documento donde se cumple y con las conclusiones que lo cierran.

Objetivo específico	**Cumplimiento**	**Concl.**
1. Marco conceptual	Ejes clasificación, optimización y representación (Tabla 4.2); feature maps (Sección 4.3.3) y QUBO/Ising (Sección 4.4.2).	2, 4
2. Pipelines de clasificación	Tres datasets y seis modelos con partición, PCA y escalado compartidos (Figura 4.6).	1–3
3. Pipelines de optimización	Tres instancias y seis solvers sobre un QUBO común (Figura 4.10).	4
4. Calidad y esfuerzo	Métricas comunes (Tablas 4.3 y 4.4), semillas fijas y tiempos de entrenamiento, predicción y resolución.	1–4
5. Interpretación en régimen NISQ	Discusión por representación (Sección 5.3) y límites del régimen.	5

Cuadro 6.1: Objetivos específicos, evidencia y conclusiones asociadas. Fuente: elaboración propia.

6.3 Cierre

La respuesta a la pregunta inicial es negativa en el rango estudiado, pero tiene una forma concreta: el modelo cuántico iguala al clásico cuando el feature map o el Hamiltoniano de coste coinciden con la estructura del dato o de la instancia, y queda por detrás cuando no coinciden. Los métodos cuánticos no fallan en sí mismos; el cuello de botella está en la representación y en el coste de simularla. La validez de esta lectura está acotada por el diseño: no se usa hardware cuántico, el número de qubits es pequeño, los hiperparámetros son fijos y los datasets son de tamaño reducido. El Capítulo 7 parte de estos límites.

7. Líneas de trabajo futuro

Las líneas de la Tabla 7.1 parten de las preguntas abiertas de la Sección 5.3 y de los límites del diseño. Las tres primeras atacan directamente esas preguntas; las tres últimas amplían el alcance. Todas cambian un único factor del protocolo y mantienen el criterio de comparación: el modelo cuántico frente al mejor clásico, con la representación como variable controlada.

Línea	**Acción concreta**	**Pregunta que responde**
Escala de la alineación	Cadenas de Ising de 6 a 12 espines con codificación directa en qubits; QSVM-ZZ frente a SVM-RBF.	¿Se mantiene la paridad observada en Ising al crecer el sistema?
Instancias QUBO mayores	TSP de 6 a 8 ciudades y MDS/MaxCut de 10 a 20 nodos de HamLib; QAOA con $p > 1$ frente a SA (Zhou et al., 2020; Sawaya et al., 2024; PennyLane, n.d.g).	¿Separa la profundidad de QAOA su rendimiento del de un annealing sobre el mismo QUBO?
Hardware real	Ejecución por shots con mitigación de errores, kernel por medidas aleatorizadas (Haug et al., 2023) y compilación estructurada (Rovara et al., 2026).	¿Qué parte del coste se debe a la simulación y cuál al método?
Representación	Basis encoding sin PCA en Bars and Stripes, re-uploading (Pérez-Salinas et al., 2020), embeddings entrenables (Lloyd et al., 2020) y elección del circuito según la complejidad del dato (de Paula Neto, 2025).	¿Puede elegirse el feature map antes de entrenar?
Entrenabilidad	Búsqueda de hiperparámetros y varianza del gradiente al crecer profundidad y qubits (McClean et al., 2018; Cerezo et al., 2025).	¿Limita el paisaje de pérdida a QNN, VQC y QAOA antes que la representación?
Otras familias	Regresión sobre Friedman #1 (Friedman, 1991; Kulkarni, n.d.), clustering (Aïmeur et al., 2007; Sentís et al., 2019; Kerenidis et al., 2019) y qPCA o autoencoders cuánticos en lugar de la PCA (Lloyd et al., 2014; Romero et al., 2017), con la salvedad de Tang (2021).	¿Se reproduce el patrón de alineación fuera de la clasificación?

Cuadro 7.1: Líneas de trabajo futuro. Fuente: elaboración propia.

Bibliografía

Aïmeur, E., Brassard, G., & Gambs, S. (2007). *Quantum clustering algorithms*. Proceedings of the 24th International Conference on Machine Learning (ICML '07), 1–8. https://doi.org/10.1145/1273496.1273497

Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J. C., Barends, R., ... Martinis, J. M. (2019). *Quantum supremacy using a programmable superconducting processor*. Nature, 574(7779), 505–510. https://doi.org/10.1038/s41586-019-1666-5

Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). *Quantum machine learning*. Nature, 549(7671), 195–202. https://doi.org/10.1038/nature23474

Bowles, J., Ahmed, S., & Schuld, M. (2024). *Better than classical? The subtle art of benchmarking quantum machine learning models*. arXiv preprint arXiv:2403.07059. https://doi.org/10.48550/arXiv.2403.07059

Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., ... Coles, P. J. (2021). *Variational quantum algorithms*. Nature Reviews Physics, 3(9), 625–644. https://doi.org/10.1038/s42254-021-00348-9

Cerezo, M., Larocca, M., García-Martín, D., Diaz, N. L., Braccia, P., Fontana, E., Rudolph, M. S., Bermejo, P., Ijaz, A., Thanasilp, S., Anschuetz, E. R., & Holmes, Z. (2025). *Does provable absence of barren plateaus imply classical simulability?* Nature Communications, 16(1), 7907. https://doi.org/10.1038/s41467-025-63099-6

CERN & Atos. (2021). *CERN–Atos Quantum Computing Fast Start* (session materials). https://indico.cern.ch/event/1098063/attachments/2357402/4023253/CERN_Atos_Fast-Start_3.pdf

Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Cortes, C., Mohri, M., & Rostamizadeh, A. (2012). *Algorithms for learning kernels based on centered alignment*. Journal of Machine Learning Research, 13, 795–828.

Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20(3), 273–297. https://doi.org/10.1007/BF00994018

Deutsch, D. (1985). *Quantum theory, the Church–Turing principle and the universal quantum computer*. Proceedings of the Royal Society of London A, 400(1818), 97–117. https://doi.org/10.1098/rspa.1985.0070

Farhi, E., Goldstone, J., & Gutmann, S. (2014). *A quantum approximate optimization algorithm*. arXiv preprint arXiv:1411.4028. https://doi.org/10.48550/arXiv.1411.4028

Fastovets, D. V., Bogdanov, Yu. I., Bantysh, B. I., & Lukichev, V. F. (2019). *Machine learning methods in quantum computing theory*. arXiv preprint arXiv:1906.10175. https://doi.org/10.48550/arXiv.1906.10175

Feynman, R. P. (1982). *Simulating physics with computers*. International Journal of Theoretical Physics, 21(6–7), 467–488. https://doi.org/10.1007/BF02650179

Friedman, J. H. (1991). *Multivariate adaptive regression splines*. The Annals of Statistics, 19(1), 1–67. https://doi.org/10.1214/aos/1176347963

Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman and Company.

Google Quantum AI and Collaborators. (2025). *Quantum error correction below the surface code threshold*. Nature, 638(8052), 920–926. https://doi.org/10.1038/s41586-024-08449-y

Grover, L. K. (1996). *A fast quantum mechanical algorithm for database search*. Proceedings of the 28th Annual ACM Symposium on Theory of Computing, 212–219. https://doi.org/10.1145/237814.237866

Gurobi Optimization. (n.d.). *Integer linear programming* (FAQ). https://www.gurobi.com/resources/faq/integer-linear-programming

Haug, T., Self, C. N., & Kim, M. S. (2023). *Quantum machine learning of large datasets using randomized measurements*. Machine Learning: Science and Technology, 4(1), 015005. https://doi.org/10.1088/2632-2153/acb0b4

Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., & Gambetta, J. M. (2019). *Supervised learning with quantum-enhanced feature spaces*. Nature, 567(7747), 209–212. https://doi.org/10.1038/s41586-019-0980-2

Huang, H.-Y., Broughton, M., Mohseni, M., Babbush, R., Boixo, S., Neven, H., & McClean, J. R. (2021). *Power of data in quantum machine learning*. Nature Communications, 12(1), 2631. https://doi.org/10.1038/s41467-021-22539-9

Jordan, M. I., & Mitchell, T. M. (2015). *Machine learning: Trends, perspectives, and prospects*. Science, 349(6245), 255–260. https://doi.org/10.1126/science.aaa8415

Kerenidis, I., Landman, J., Luongo, A., & Prakash, A. (2019). *q-means: A quantum algorithm for unsupervised machine learning*. Advances in Neural Information Processing Systems, 32. https://doi.org/10.48550/arXiv.1812.03584

Kulkarni, A. (n.d.). *Quantum-Linear-Regression* (software repository). GitHub. https://github.com/amey-kulkarni27/Quantum-Linear-Regression

Liu, Y., Arunachalam, S., & Temme, K. (2021). *A rigorous and robust quantum speed-up in supervised machine learning*. Nature Physics, 17(9), 1013–1017. https://doi.org/10.1038/s41567-021-01287-z

Lloyd, S., Mohseni, M., & Rebentrost, P. (2014). *Quantum principal component analysis*. Nature Physics, 10(9), 631–633. https://doi.org/10.1038/nphys3029

Lloyd, S., Schuld, M., Ijaz, A., Izaac, J., & Killoran, N. (2020). *Quantum embeddings for machine learning*. arXiv preprint arXiv:2001.03622. https://doi.org/10.48550/arXiv.2001.03622

Lubinski, T., Coffrin, C., McGeoch, C., Sathe, P., Apanavicius, J., Bernal Neira, D., & QED-C Collaboration. (2024). *Optimization applications as quantum performance benchmarks*. ACM Transactions on Quantum Computing, 5(3). https://doi.org/10.1145/3678184

Lucas, A. (2014). *Ising formulations of many NP problems*. Frontiers in Physics, 2, 5. https://doi.org/10.3389/fphy.2014.00005

McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., & Neven, H. (2018). *Barren plateaus in quantum neural network training landscapes*. Nature Communications, 9(1), 4812. https://doi.org/10.1038/s41467-018-07090-4

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511976667

Pan, F., Chen, K., & Zhang, P. (2022). *Solving the sampling problem of the Sycamore quantum circuits*. Physical Review Letters, 129(9), 090502. https://doi.org/10.1103/PhysRevLett.129.090502

Parate, A. (2025). *A complete guide to quantum data encoding*. Medium. https://medium.com/@parateaadish/a-complete-guide-to-quantum-data-encoding-786554b346ab

Parzygnat, A. J., Bradley, T.-D., Vlasic, A., & Pham, A. (2025). *Toward structure-preserving quantum encodings*. Physical Review Research. https://doi.org/10.1103/rph8-g15q

de Paula Neto, F. M. (2025). *Data complexity measures for quantum circuits architecture recommendation*. arXiv preprint arXiv:2502.15129. https://doi.org/10.48550/arXiv.2502.15129

PennyLane / Xanadu. (n.d.). *Bars and Stripes* (dataset). https://pennylane.ai/datasets/bars-and-stripes

PennyLane / Xanadu. (n.d.). *Conventional datasets for QML*. https://pennylane.ai/datasets/conventional-datasets-for-qml

Schoch, I. / PennyLane. (2022). *How to embed data into a quantum state*. PennyLane Blog. https://pennylane.ai/blog/2022/08/how-to-embed-data-into-a-quantum-state/

PennyLane / Xanadu. (n.d.). *Quantum embedding* (glossary). https://pennylane.ai/glossary/quantum_embedding

PennyLane / Xanadu. (n.d.). *Grover's algorithm* (demo). https://pennylane.ai/demos/tutorial_grovers_algorithm

PennyLane / Xanadu. (n.d.). *HamLib MaxCut* (dataset). https://pennylane.ai/datasets/hamlib-maxcut

PennyLane / Xanadu. (n.d.). *HamLib travelling salesperson problem* (dataset). https://pennylane.ai/datasets/hamlib-travelling-salesperson-problem

PennyLane / Xanadu. (n.d.). *Ising* (dataset / qspin). https://pennylane.ai/datasets/ising

PennyLane / Xanadu. (n.d.). *Intro to QAOA* (demo). https://pennylane.ai/demos/tutorial_qaoa_intro

Lowe, A., & Soni, J. / PennyLane. (2019). *QAOA for MaxCut* (demo). https://pennylane.ai/demos/tutorial_qaoa_maxcut

PennyLane / Xanadu. (n.d.). *QFT and classical groups* (demo). https://pennylane.ai/demos/tutorial_qft_and_groups

PennyLane / Xanadu. (n.d.). *Quantum transfer learning* (demo). https://pennylane.ai/demos/tutorial_quantum_transfer_learning

PennyLane / Xanadu. (n.d.). *Variational classifier* (demo). https://pennylane.ai/demos/tutorial_variational_classifier

PennyLane / Xanadu. (n.d.). *What is a quantum dataset?* https://pennylane.ai/datasets/what-is-a-quantum-dataset

Pérez-Salinas, A., Cervera-Lierta, A., Gil-Fuster, E., & Latorre, J. I. (2020). *Data re-uploading for a universal quantum classifier*. Quantum, 4, 226. https://doi.org/10.22331/q-2020-02-06-226

Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., ... O'Brien, J. L. (2014). *A variational eigenvalue solver on a photonic quantum processor*. Nature Communications, 5(1), 4213. https://doi.org/10.1038/ncomms5213

Preskill, J. (2018). *Quantum computing in the NISQ era and beyond*. Quantum, 2, 79. https://doi.org/10.22331/q-2018-08-06-79

Qiskit contributors. (2026). *Qiskit: An open-source framework for quantum computing* (Versions 2.3.0 and 2.6.1). IBM Quantum. https://doi.org/10.5281/zenodo.2573505

Quantum Zeitgeist. (n.d.). *PennyLane and quantum machine learning* (hybrid quantum-classical algorithms). https://quantumzeitgeist.com/pennylane-and-quantum-machine-learning/#hybrid-quantum-classical-algorithms

Quantum Zeitgeist. (n.d.). *Understanding quantum algorithms: A beginner's guide.* https://quantumzeitgeist.com/understanding-quantum-algorithms-a-beginners-guide/

Quantum Zeitgeist. (n.d.). *FBI battles foreign adversaries targeting U.S. quantum tech.* https://quantumzeitgeist.com/fbi-battles-foreign-adversaries-targeting-u-s-quantum-tech-bolstering-national-security/

Romero, J., Olson, J. P., & Aspuru-Guzik, A. (2017). *Quantum autoencoders for efficient compression of quantum data.* Quantum Science and Technology, 2(4), 045001. https://doi.org/10.1088/2058-9565/aa8072

Rovara, D., Haag, D., Koch, M., Izaac, J., Sivarajah, S., Wille, R., … Ittah, D. (2026). *Why are we unrolling? The importance of structured quantum programs for compilation.* arXiv preprint arXiv:2609.16171. https://doi.org/10.48550/arXiv.2609.16171

Sawaya, N. P. D., Marti-Dafcik, D., Ho, Y., Tabor, D. P., Bernal Neira, D. E., Magann, A. B., … Camps, D. (2024). *HamLib: A library of Hamiltonians for benchmarking quantum algorithms and hardware.* arXiv preprint arXiv:2306.13126. https://doi.org/10.48550/arXiv.2306.13126

Schuld, M. (2021). *Supervised quantum machine learning models are kernel methods.* arXiv preprint arXiv:2101.11020. https://doi.org/10.48550/arXiv.2101.11020

Schuld, M., & Killoran, N. (2019). *Quantum machine learning in feature Hilbert spaces.* Physical Review Letters, 122(4), 040504. https://doi.org/10.1103/PhysRevLett.122.040504

scikit-learn developers. (n.d.). *The Iris Dataset* (example). https://scikit-learn.org/1.5/auto_examples/datasets/plot_iris_dataset.html

Sentís, G., Monràs, A., Muñoz-Tapia, R., Calsamiglia, J., & Bagan, E. (2019). *Unsupervised classification of quantum data.* Physical Review X, 9(4), 041029. https://doi.org/10.1103/PhysRevX.9.041029

Shor, P. W. (1994). *Algorithms for quantum computation: Discrete logarithms and factoring.* Proceedings of the 35th Annual Symposium on Foundations of Computer Science, 124–134. https://doi.org/10.1109/SFCS.1994.365700

Sim, S., Johnson, P. D., & Aspuru-Guzik, A. (2019). *Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms.* Advanced Quantum Technologies, 2(12), 1900070. https://doi.org/10.1002/qute.201900070

Somma, R. D., Boixo, S., & Barnum, H. (2008). *Quantum simulated annealing.* arXiv preprint arXiv:0712.1008. https://doi.org/10.48550/arXiv.0712.1008

Tang, E. (2021). *Quantum principal component analysis only achieves an exponential speedup because of its state preparation assumptions.* Physical Review Letters, 127(6), 060503. https://doi.org/10.1103/PhysRevLett.127.060503

Thanasilp, S., Wang, S., Cerezo, M., & Holmes, Z. (2024). *Exponential concentration in quantum kernel methods.* Nature Communications, 15(1), 5200. https://doi.org/10.1038/s41467-024-49287-w

Vikstål, P., Grönkvist, M., Svensson, M., Andersson, M., Johansson, G., & Ferrini, G. (2020). *Applying the quantum approximate optimization algorithm to the tail-assignment problem.* Physical Review Applied, 14(3), 034009. https://doi.org/10.1103/PhysRevApplied.14.034009

Vocal Media. (n.d.). *Quantum algorithms outperform classical solvers in optimization tasks, study reveals.* https://vocal.media/education/quantum-algorithms-outperform-classical-solvers-in-optimization-tasks-study-reveals

Wang, G. (n.d.). *Classical versus quantum machine learning: A comparative study.* Medium. https://medium.com/@gwrx2005/classical-versus-quantum-machine-learning-a-comparative-study-1e65ced52a42

Wolpert, D. H., & Macready, W. G. (1997). *No free lunch theorems for optimization.* IEEE Transactions on Evolutionary Computation, 1(1), 67–82. https://doi.org/10.1109/4235.585893

XanaduAI. (n.d.). *qml-benchmarks* (software repository). GitHub. https://github.com/XanaduAI/qml-benchmarks

Yu, C., Liu, J., Ge, Z., Wu, X., Zhu, L., Zhao, P., & Zhao, J. (2026). *Quantum vs. classical machine learning: A unified empirical comparison.* arXiv preprint arXiv:2607.01197. https://doi.org/10.48550/arXiv.2607.01197

Z-537-437. (n.d.). *QML* (software repository accompanying Yu et al.). GitHub. https://github.com/Z-537-437/QML

Zhou, L., Wang, S.-T., Choi, S., Pichler, H., & Lukin, M. D. (2020). *Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices.* Physical Review X, 10(2), 021067. https://doi.org/10.1103/PhysRevX.10.021067

A. Apéndices

A.1 Repositorio y catálogo de notebooks

El código, los datos de entrada, las métricas y las figuras del trabajo están en el repositorio público:

`https://github.com/juandapradam12/UNIR-MasterQC-QuantumAdvantageInDataScience`

La Tabla A.1 resume la organización del corpus experimental, repartido entre las carpetas `Notebooks/Classification/` y `Notebooks/Optimization/`. Cada dataset o instancia tiene un notebook por modelo o solver, y una carpeta de resultados agrega las métricas y genera las figuras comparativas del Capítulo 5.

Carpeta	**Contenido**	**Datos**
`IrisClassification/`	SVM, RF, XGBoost, QSVM, QNN, VQC	Iris
`BSClassification/`	mismos seis modelos	Bars and Stripes
`IsingClassification/`	mismos seis modelos	Ising 1D
`ResultsClassification/`	agregación de métricas y figuras comparativas	tres datasets
`TSPOptimization/`	fuerza bruta, greedy, ILP, QUBO+SA, QAOA, VQE	TSP
`MinimumDominating-SetOptimization/`	mismos seis solvers	MDS
`MaxCutOptimization/`	mismos seis solvers	MaxCut (HamLib)
`ResultsOptimization/`	agregación de métricas y figuras comparativas	tres instancias

Cuadro A.1: Organización del corpus experimental. Fuente: elaboración propia.

A.2 Entorno de software

La Tabla A.2 recoge las versiones fijadas en `requirements.txt`, con las que se ejecutaron todos los notebooks. Toda la computación cuántica se realiza en simulación de estado sobre CPU.

Paquete	Versión	Paquete	Versión
PennyLane	0.45.1	scikit-learn	1.9.1
PennyLane Lightning	0.45.0	XGBoost	3.1.2
Qiskit (Qiskit contributors, 2026)	2.6.1	PuLP (CBC)	2.9.0
Qiskit Aer	0.18.2	NetworkX	3.7
PyTorch	2.15.0	NumPy	2.5.4
SciPy	1.18.1	pandas	3.1.0
h5py	3.16.0	matplotlib	3.11.2

Cuadro A.2: Versiones de las dependencias principales. Fuente: elaboración propia.

A.3 Datasets e instancias

La Tabla A.3 resume el tamaño de cada dataset e instancia y la partición usada. En clasificación, la partición es estratificada con 40 % de test y semilla 123; la varianza explicada se refiere a la PCA a cuatro componentes.

Dataset / instancia	Tamaño	Origen	Preparación
Iris	150 muestras, $d = 4$, 3 clases (90/60)	scikit-learn (scikit-learn developers, n.d.)	sin PCA
Bars and Stripes	1200 muestras, $d = 16$, 2 clases (720/480)	PennyLane, 4×4 (PennyLane, n.d.a)	PCA a 4 (57.5 % var.)
Ising 1D	200 muestras, $d = 32$, 2 fases (120/80)	PennyLane `qspin`, cadenas abierta y cerrada de 4 espines (PennyLane, n.d.h)	PCA a 4 (99.9 % var.)
TSP	5 nodos, grafo completo	instancia propia	QUBO
MDS	6 vértices, 6 aristas	instancia propia	QUBO con penalización
MaxCut	6 vértices, 12 aristas	HamLib (Sawaya et al., 2024)	Ising sin restricciones

Cuadro A.3: Datasets de clasificación (train/test entre paréntesis) e instancias de optimización. Fuente: elaboración propia.

A.4 Figuras complementarias de clasificación

Las Figuras A.1 a A.4 agregan los tres datasets de clasificación y complementan los resultados por dataset del Capítulo 5. Los valores medios se calculan sobre Iris, Bars and Stripes e Ising con el mismo peso.

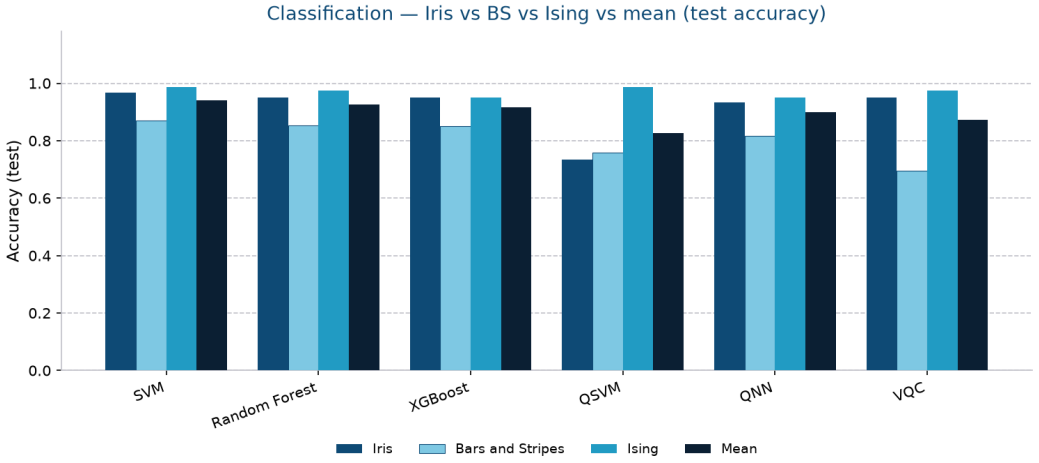


Figura A.1: Accuracy en test por modelo y dataset, con la media de los tres. QSVM y VQC son los modelos con mayor variación entre datasets.

Fuente: elaboración propia.

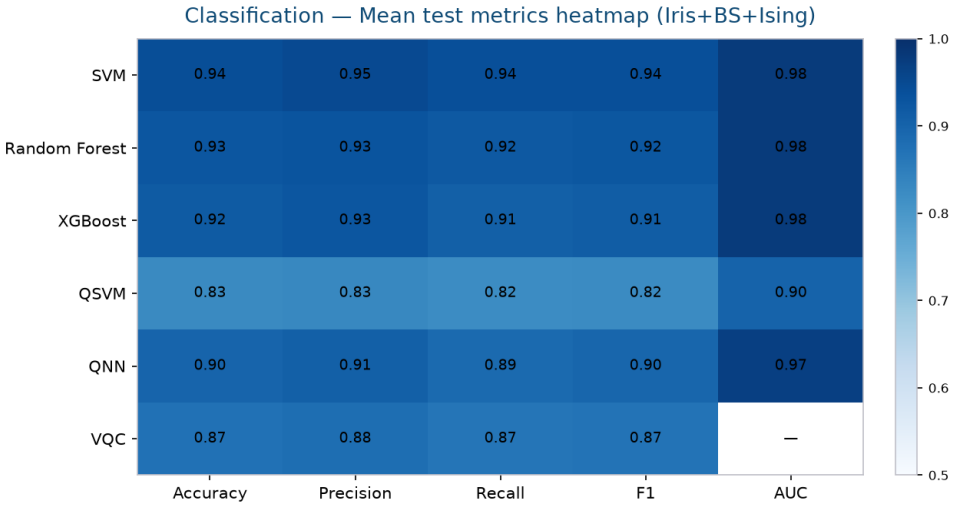


Figura A.2: Métricas de test medias por modelo. El VQC no exporta probabilidades, por lo que no tiene AUC.

Fuente: elaboración propia.

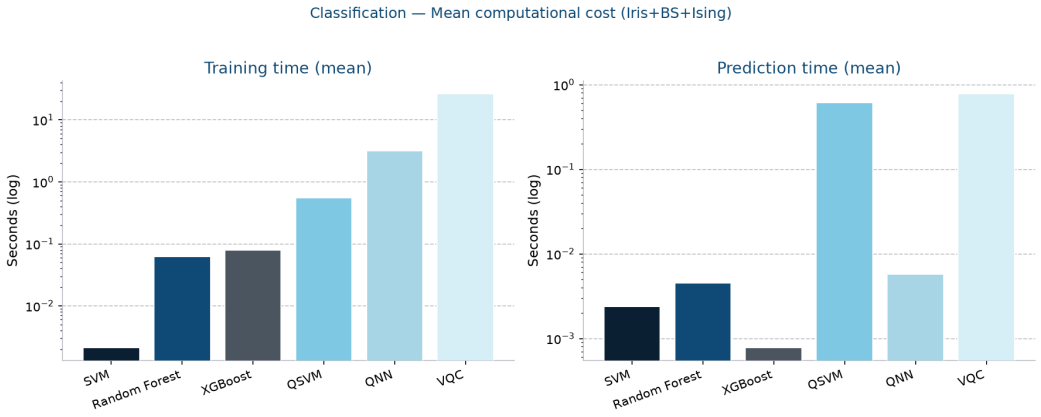


Figura A.3: Tiempos medios de entrenamiento y predicción por modelo (escala logarítmica).

Fuente: elaboración propia.

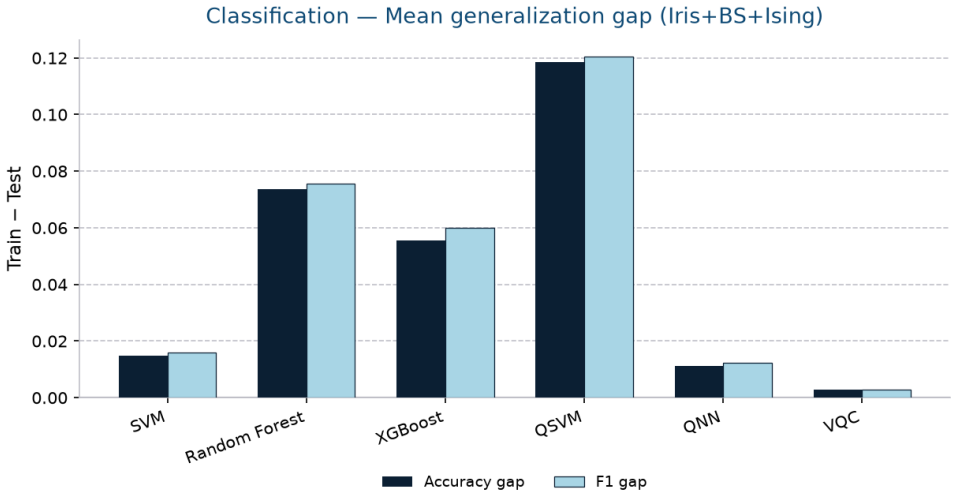


Figura A.4: Brecha de generalización media (train − test) en accuracy y F1.

Fuente: elaboración propia.